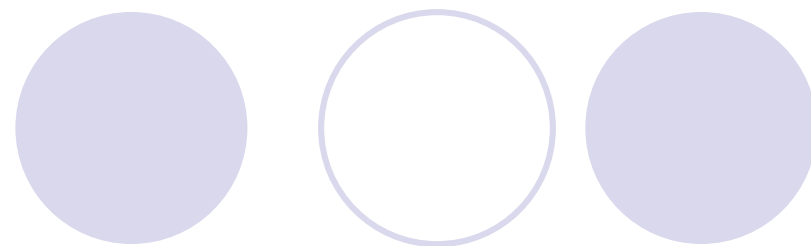


第3章 存储系统



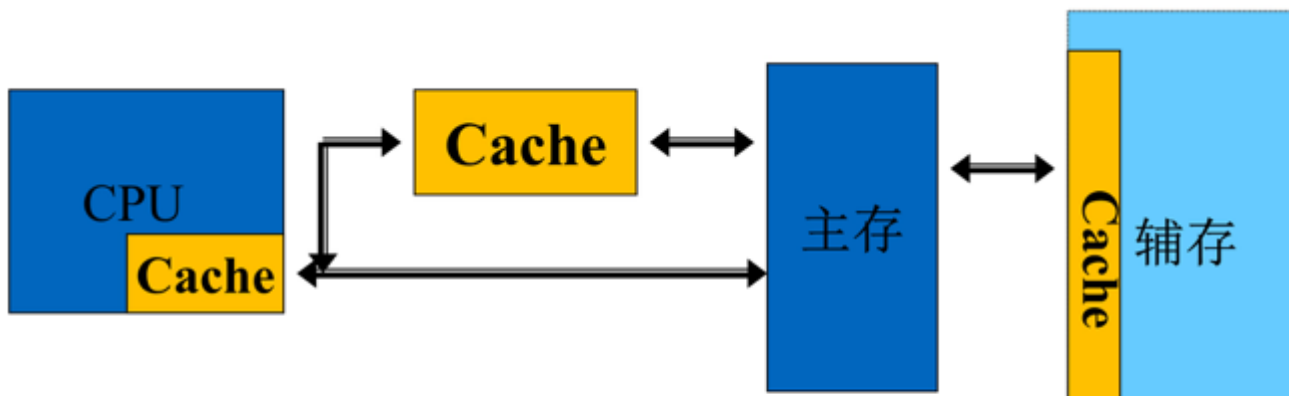
- 存储器概述（了解）
- 随机存储器-存储器容量扩展（重点）
- 只读存储器（了解）
- 并行存储器（熟悉）
- **cache**存储器（重点和难点）
- 虚拟存储器（重点）

3.6 Cache存储器(重点、难点)

- Cache基本原理
- 主存与Cache的地址映射
- Cache的替换算法
- Cache的写操作策略

3.6 Cache存储器(重点、难点)

存储器：速度 容量 价格



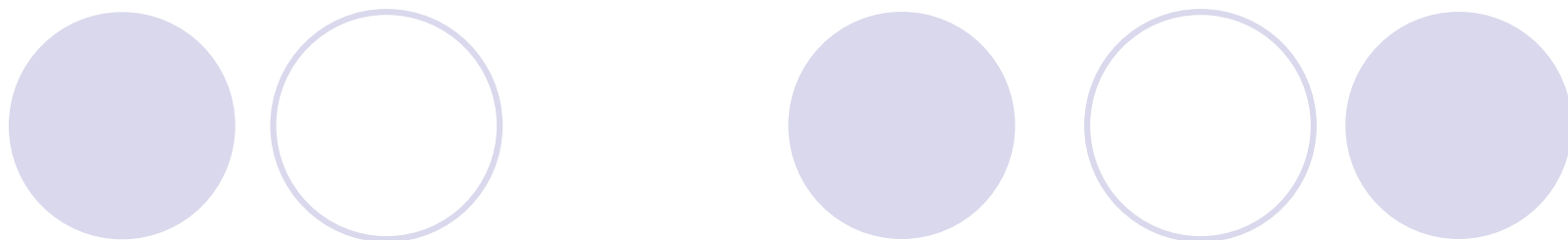
- Cache的功能:

缓解快速CPU与慢速的主存之间的速度差异

- Cache的理论基础:

局部性原理

空间局部性 + 时间局部性



例 局部性分析。以下程序A和B中，哪一个对数组A[2048][2048]引用的**空间局部性**更好？**时间局部性**呢？变量sum的空间局部性和时间局部性如何？

假定数组在主存按行优先顺序存放

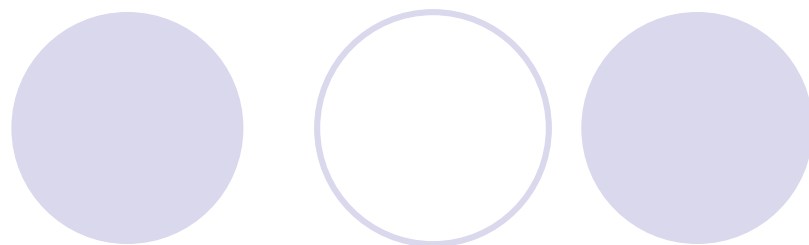
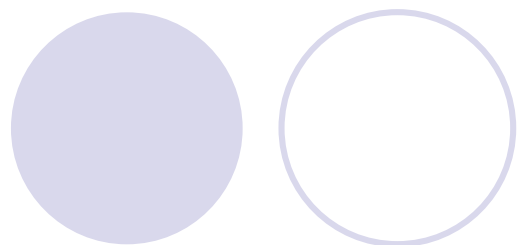


程序段A

```
Int sumarryrows (int A[M][N])  
{  
    int i, j, sum = 0;  
    for ( i=0; i < M; i++)  
        for ( j=0; j < M; j++)  
            sum += A[i][j];  
    return sum;  
}
```

程序段B

```
Int sumarrycols (int A[M][N])  
{  
    int i, j, sum = 0;  
    for ( j=0; j < M; j++)  
        for ( i=0; i < M; i++)  
            sum += A[i][j];  
    return sum;  
}
```



例 局部性分析。以下程序A和B中，哪一个对数组A[2048][2048]引用的**空间局部性**更好？**时间局部性**呢？变量sum的空间局部性和时间局部性如何？

时间局部性的程序结构体现



循环



循环结构中的变量具有**时间局部性**

空间局部性的程序结构体现



顺序



顺序访问特征变量具有**空间局部性**



例 局部性分析。以下程序A和B中，哪一个对数组A[2048][2048]引用的**空间局部性**更好？**时间局部性**呢？变量sum的空间局部性和时间局部性如何？

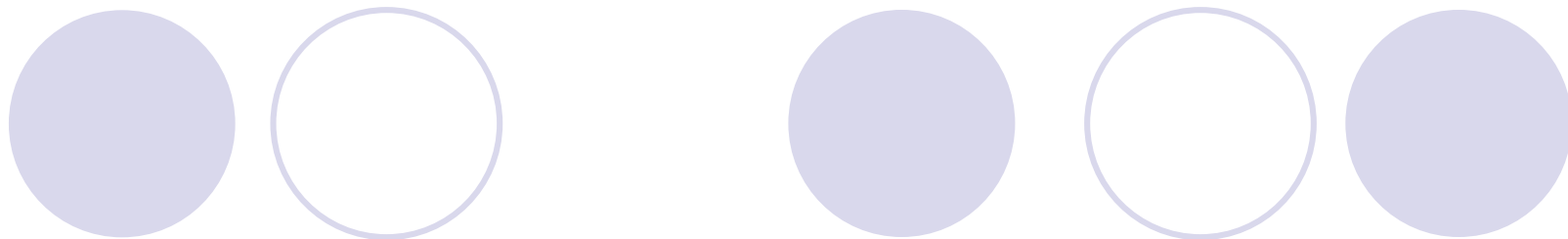
假定数组在主存按行优先顺序存放



程序段A

```
Int sumarryrows (int A[M][N])  
{  
    int i, j, sum = 0;  
    for ( i=0; i < M; i++)  
        for ( j=0; j < M; j++)  
            sum += A[i][j];  
    return sum;  
}
```

- 数组访问顺序与存放顺序一致，空间局部性好！
- 数组虽在循环体中，但每个元素只用一次，时间局部性差！
- 变量sum在循环体中，时间局部性好！
- 变量sum只有一个变量，空间局部性差！



例 局部性分析。以下程序A和B中，哪一个对数组A[2048][2048]引用的**空间局部性**更好？**时间局部性**呢？变量sum的空间局部性和时间局部性如何？

假定数组在主存按行优先顺序存放

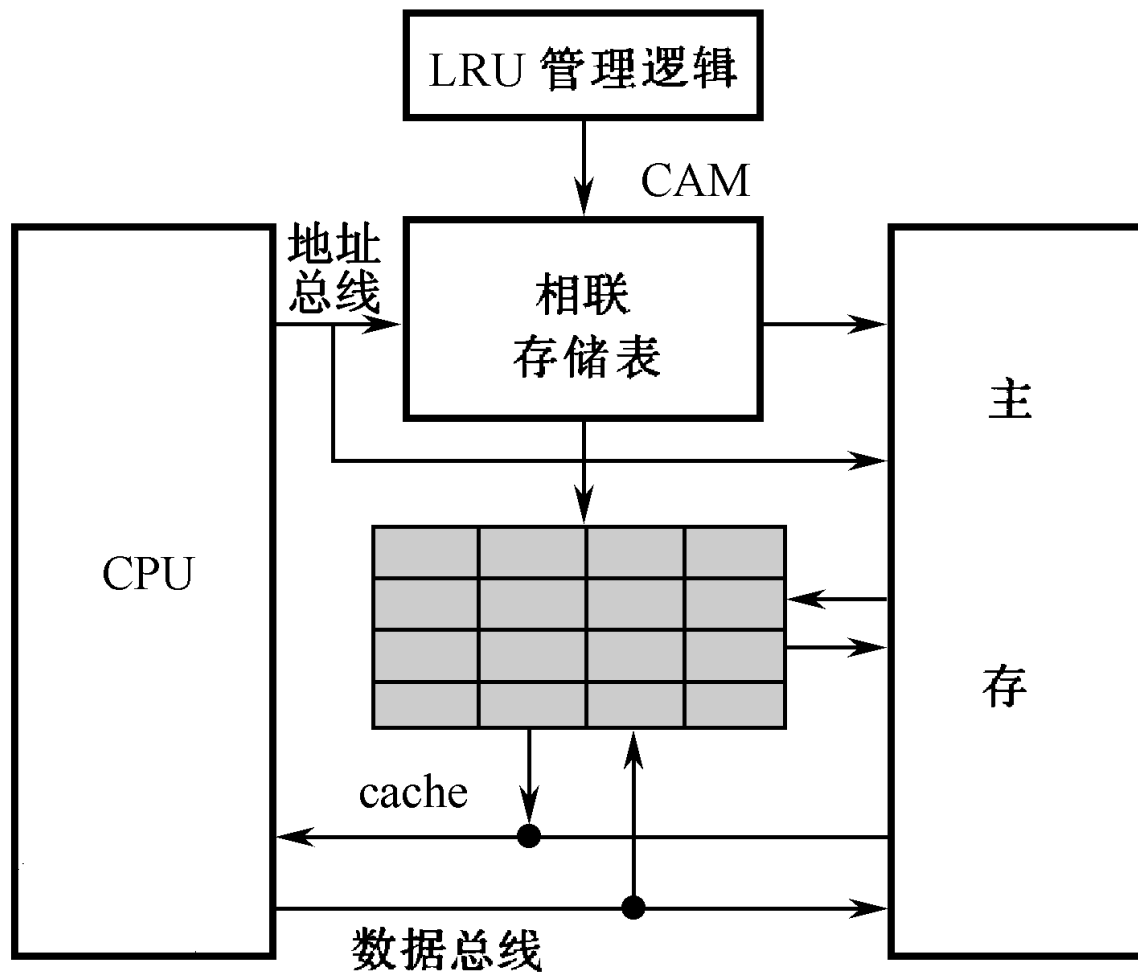


程序段B

```
Int sumarrycols (int A[M][N])  
{  
    int i, j, sum = 0;  
    for (j=0; j < M; j++)  
        for (i=0; i < M; i++)  
            sum += A[i][j];  
    return sum;  
}
```

- 数组访问顺序与存放顺序不一致，空间局部性差！
- 数组虽在循环体中，但每个元素只用一次，时间局部性差！
- 变量sum在循环体中，时间局部性好！
- 变量sum只有一个变量，空间局部性差！

3.6.1 Cache基本原理



3.6.1 Cache基本原理



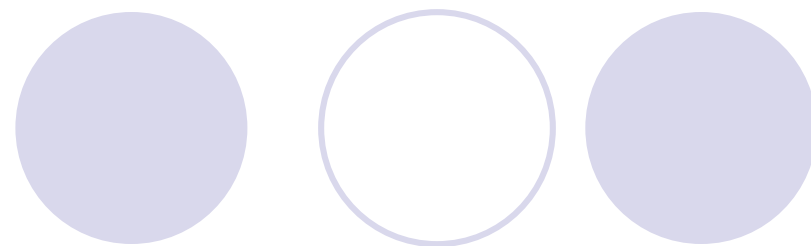
Cache的命中率

- 增加**Cache**的目的，就是在性能上使主存的平均读出时间尽可能接近**Cache**的读出时间。因此,**Cache**的命中率应接近于1。

$$h = \frac{N_c}{N_c + N_m}$$

$$t_a = h t_c + (1 - h) t_m$$

Cache的命中率



- 设 $r=t_m/t_c$ 表示主存慢于Cache的倍率, e 表示访问效率:

$$e = \frac{t_c}{t_a} = \frac{t_c}{ht_c + (1-h)t_m} = \frac{1}{h + (1-h)r}$$

- 由表达式看出, 为提高访问效率, 命中率 h 越接近1越好, r 值以5-10为宜, 不宜太大。
- 命中率 h 与程序的行为、Cache的容量、组织方式、块的大小有关。

例题：

- CPU执行一段程序时，**Cache**完成存取的次数为**1900次**，**主存**完成存取的次数为**100次**，已知**Cache**存取周期为**50ns**，**主存**存取周期为**250ns**，求**Cache/主存系统**的**平均访问时间**和**效率**。

$$t_a = ht_c + (1 - h)t_m = \frac{1900}{1900 + 100} * 50 + \frac{100}{1900 + 100} * 250 = 60ns$$

$$e = \frac{t_c}{t_a} = \frac{50}{60} = 0.8333$$

Cache的设计原则

➤ **命中率** 希望cache的实际命中率接近于1

➤ **透明性** 希望cache对CPU而言是透明的

□ 不论是否有cache，CPU访存的方法都是一样的，软件不需增加任何指令就可以访问cache

☒ 不提高软件复杂度

☒ 硬件支持保证速度足够快

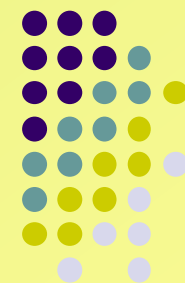
➤ 解决了命中率和透明性问题，就CPU访存的角度而言，内存将具有主存的容量和接近cache的速度

➤ 必须增加一定的硬件电路完成控制功能

cache结构设计必须解决的问题

- 如何存放，如何访问，如何替换，如何改写？
- 地址映射和地址变换：
 - ❑ 如何将主存定位在cache中
 - ❑ 如何将主存地址变换为cache地址
- 替换：在cache不命中时如何替换cache中的内容
- 更新：如何保持主存与cache的一致性

3.6.2主存与Cache的地址映射

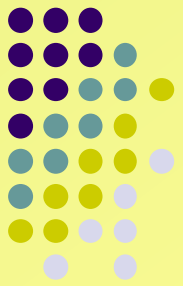


➤ 问题引入：

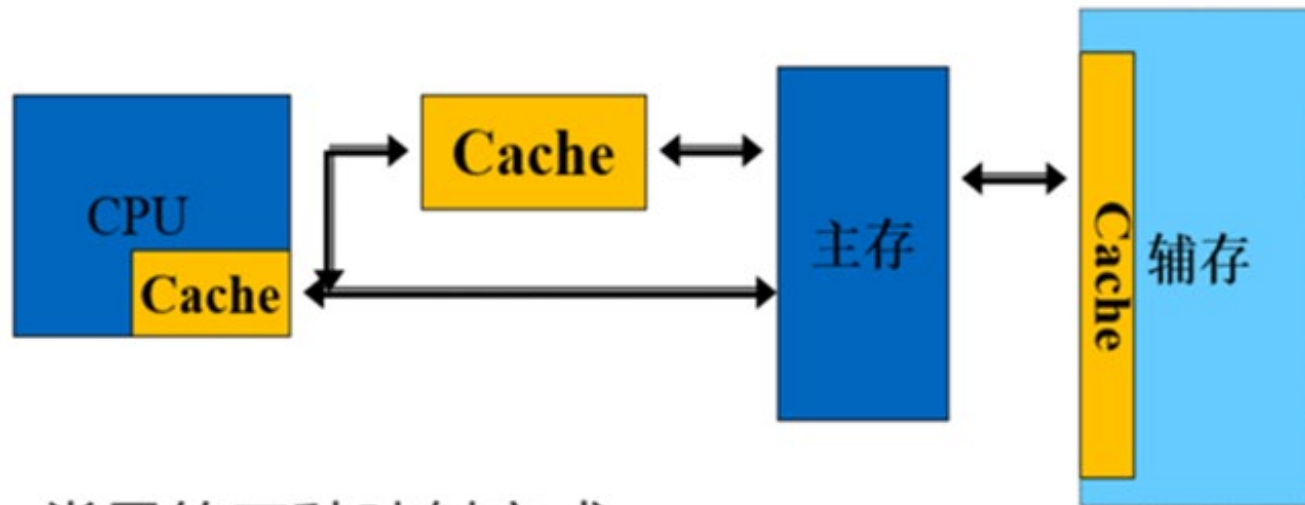
- › 思考：CPU要取一个字，肯定要给出这个字所在内存的地址，然后将该地址发往Cache，判断所要取的字是否在Cache中，在就取走，不在就去内存找。



- 必须应用某种方法，把主存的地址定位到 **Cache** 中的确切地址——地址映射
- 无论选择那种映射方式，都要把主存和 **Cache** 划分为同样大小的“块”。
- 硬件实现，透明性



3.6.2主存与Cache的地址映射



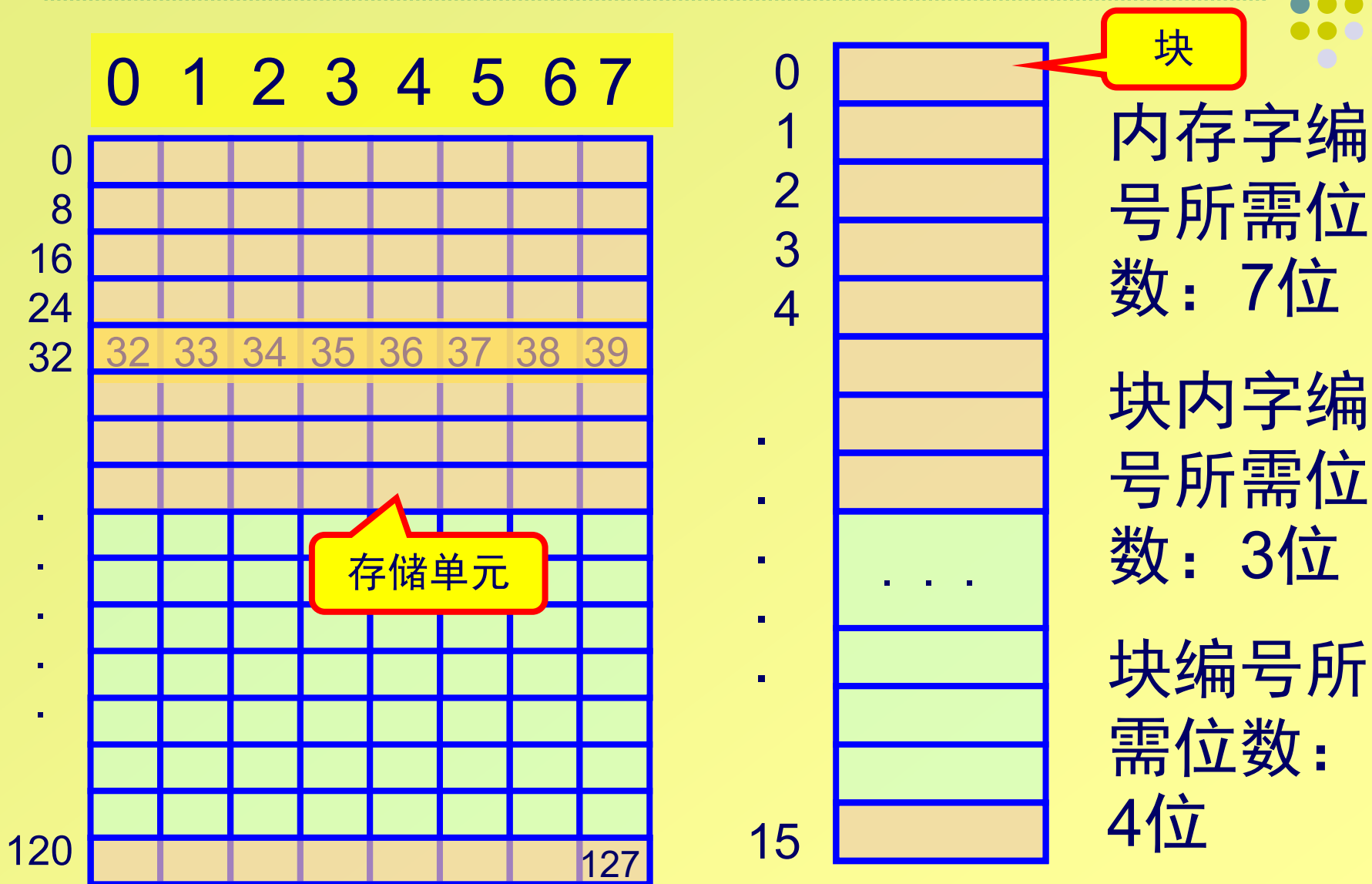
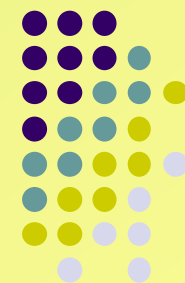
- 常见的三种映射方式

全相联 (fully-associated)

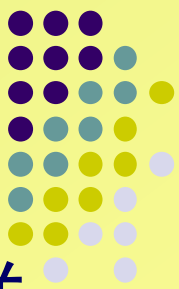
直接相联 (direct mapped)

组相联 (set-associated)

先明确一个简单的情况

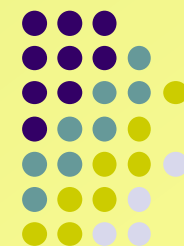


先明确一个简单的情况



- 当前知道了某个存储单元的地址35，怎样计算它所在的块编号？
 - › 因为8个存储单元一个块，所以
 - › $35 / 8 = 4 \cdots 3$
 - › 即35号存储单元位于第4块的第3号字

先明确一个简单的情况



		000 010 100 110							
0000	000								
0001	000								
0010	000								
0011	000								
0100	000	32	33	34	35	36	37	38	39
.									
.									
.									
.									
.									
.									
1111	000								127

存储单元

块地址4位

字地址3位

0000

0001

0010

0011

0100

.

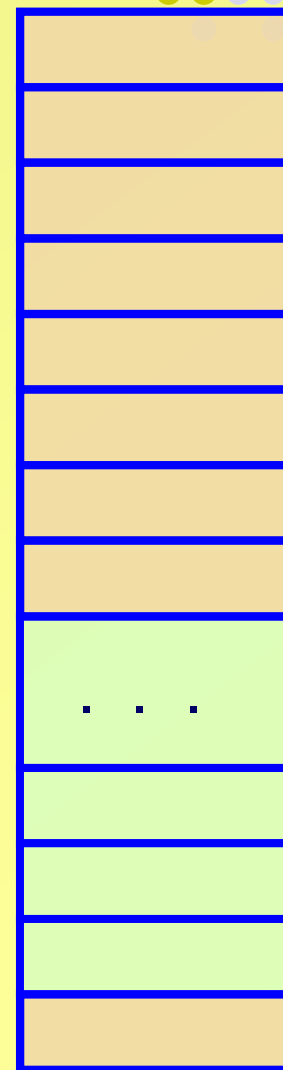
.

.

.

.

1111



XXXX XXXX | XX

块内地址

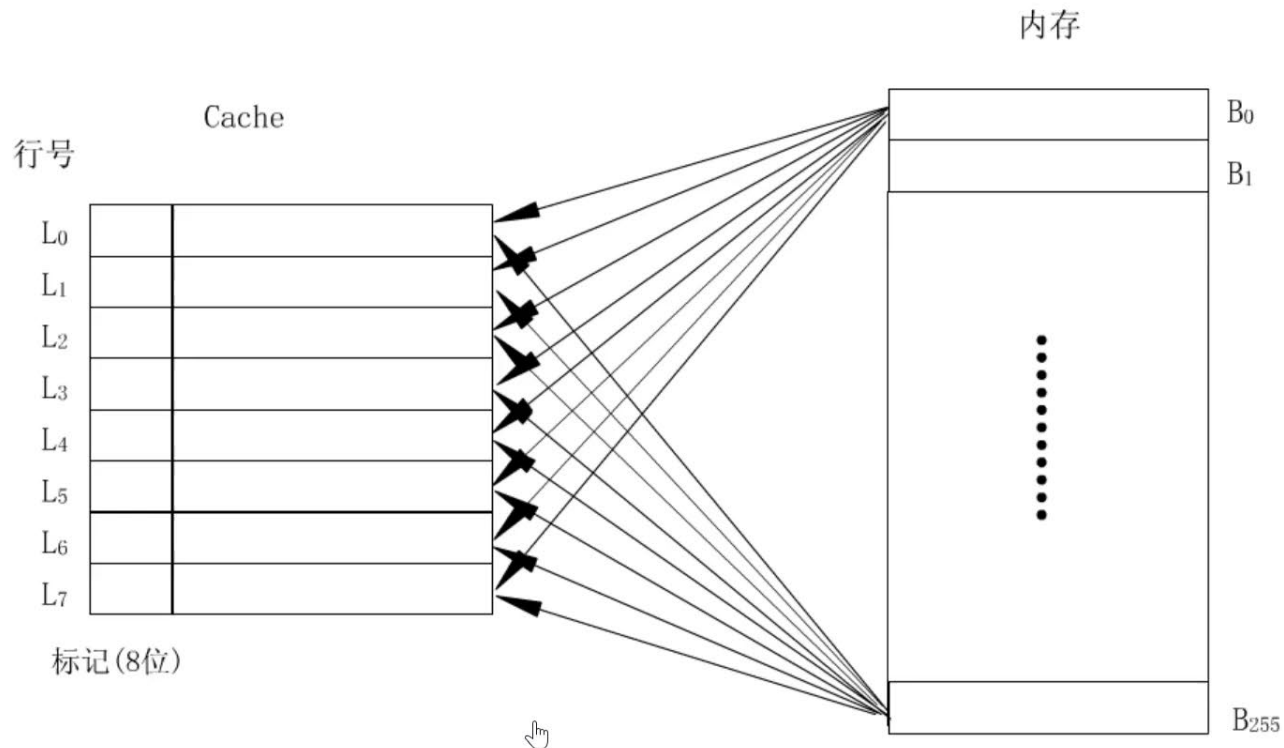
每行(块)的字数相同



$2^w \uparrow$: $4 \uparrow$

一、全相联的映射方式

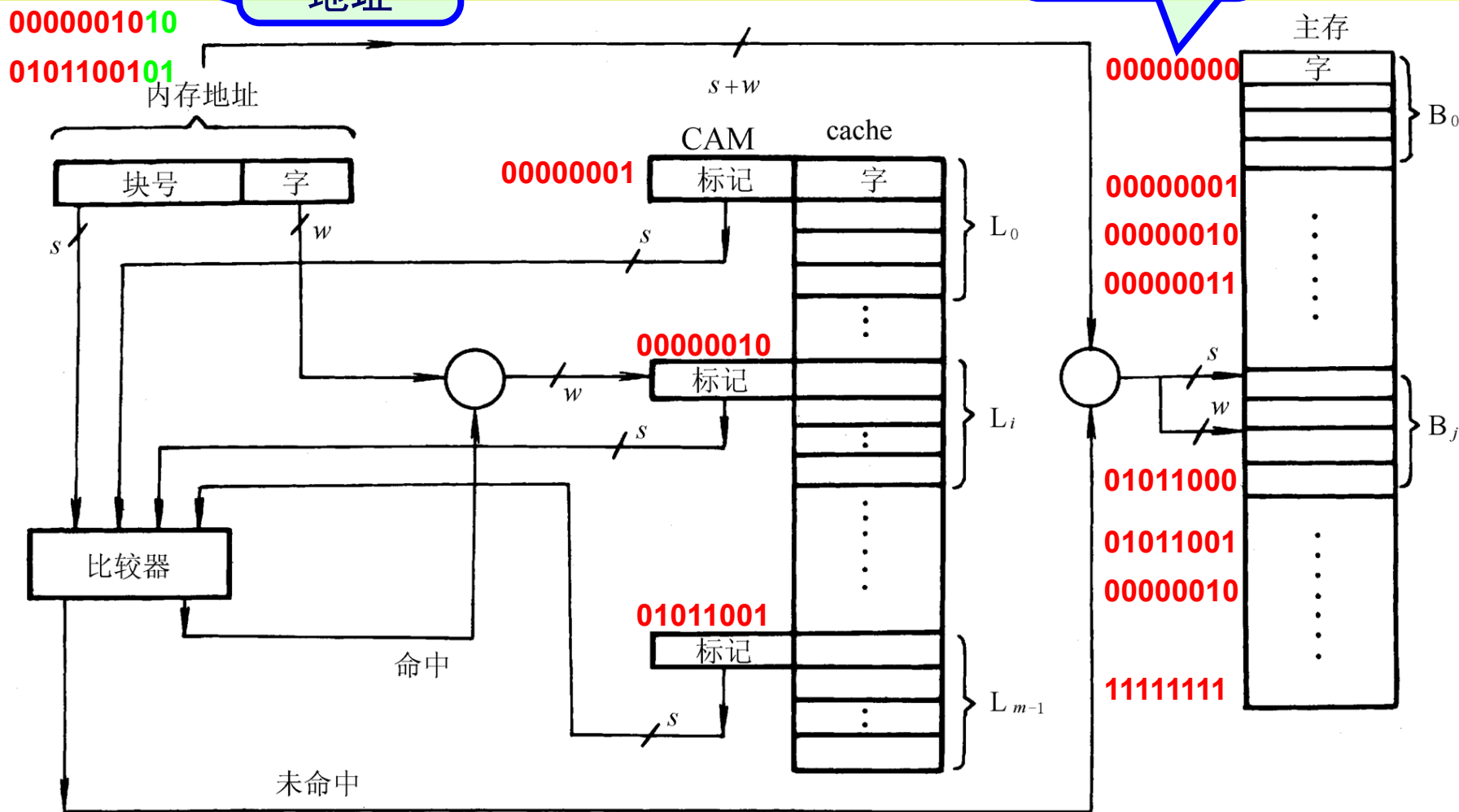
图3.33 全相联映射的Cache组织



一、全相联的映射方式

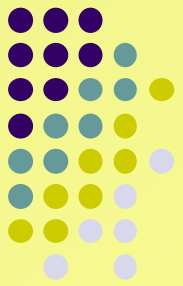
完整内存地址

块地址
(块号)



(b) 全相联cache的检索过程

一、全相联的映射方式



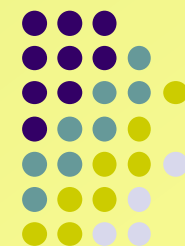
特点：

- › 优点：冲突概率小，Cache的利用高。
- › 缺点：比较电路实现成本高，需要一个访问速度很快代价高的相联存储器

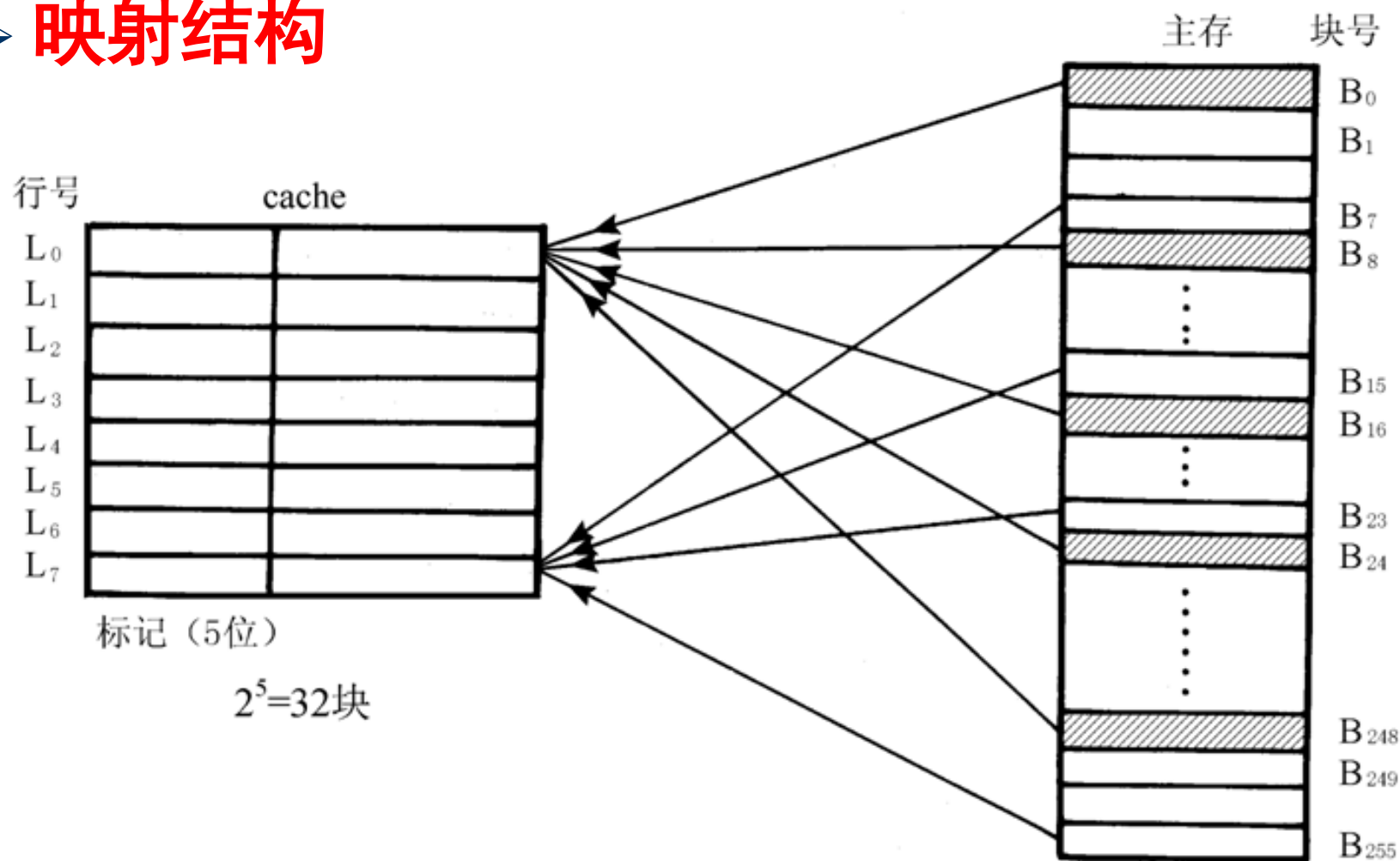
应用场合：

- › 适用于小容量的Cache

二、直接映射方式



➤ 映射结构



(a) 直接映射示意图

二、直接映射方式

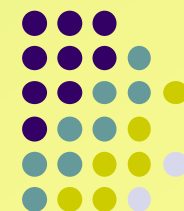
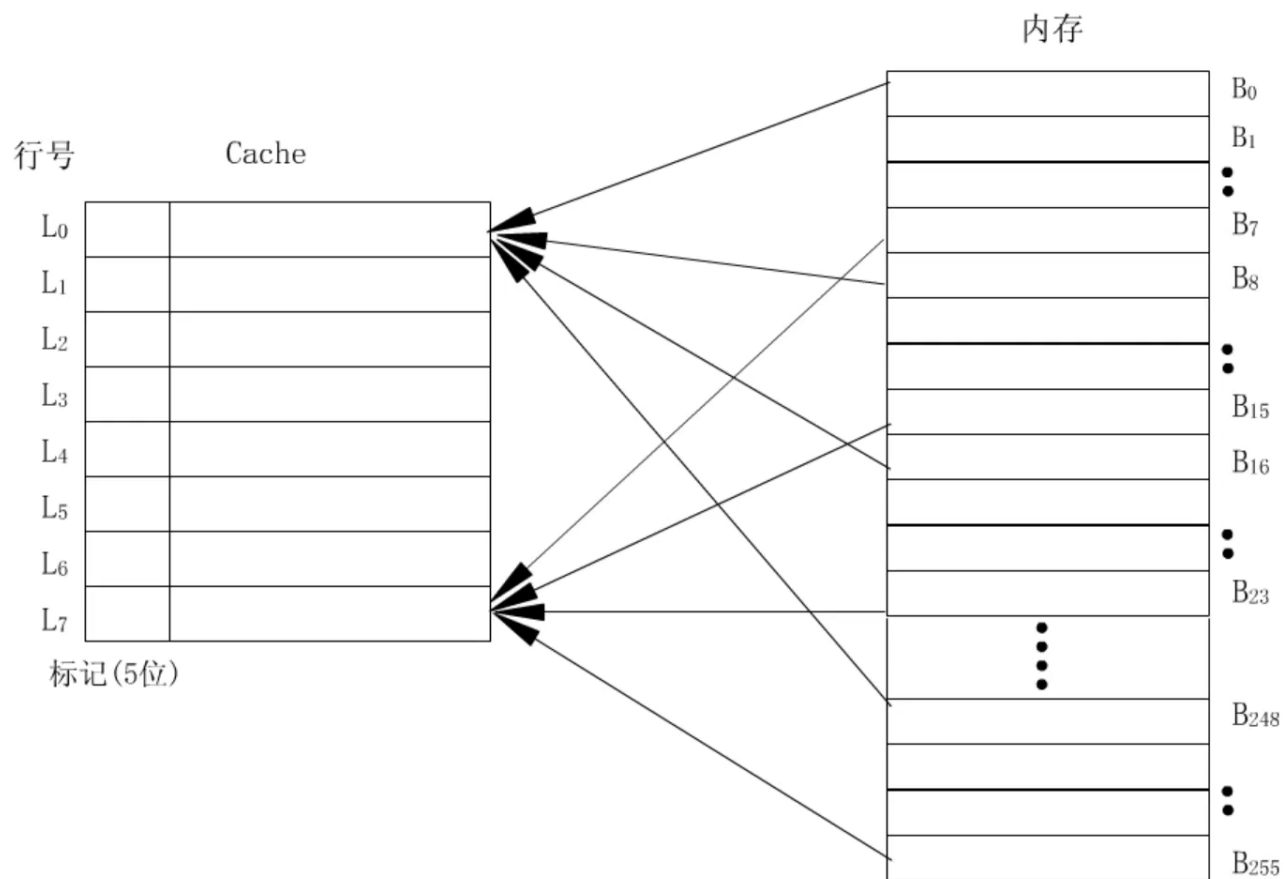


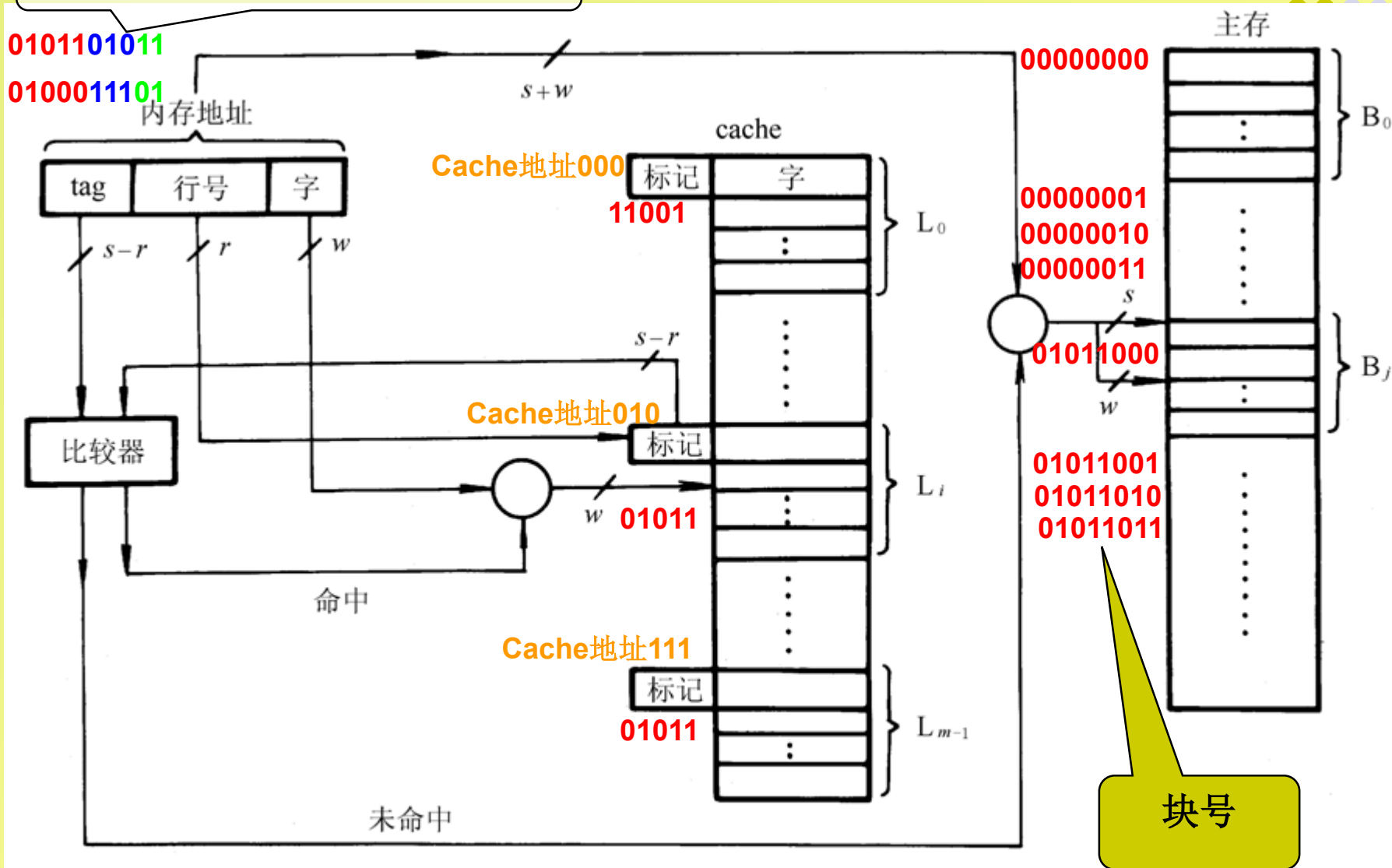
图3.34 直接相联映射的Cache组织



二、直接映射方式

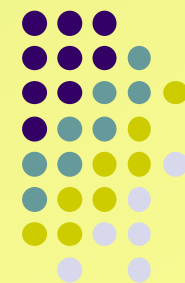
映射检索过程

蓝色：行号；绿色：字地址



(b) 直接映射cache的检索过程

二、直接映射方式



特点：

- › 优点：硬件实现简单
- › 缺点：冲突概率高（易抖动，频繁交换）

应用场合：

- › 适合大容量Cache



例题：

- 设主存容量**1MB**，Cache容量**16KB**，块的大小为**512B**
 - (1) 写出Cache地址格式；
 - (2) 写出主存地址格式；
 - (3) 块表容量为多少；
 - (4) 画出直接方式地址映象及变换示意图。

答案:

- Cache容量**16KB**，所以**Cache地址**共**14位**；
块的大小为**512B**，所以**Cache的行内地址**
9位，**行地址5位**，行数为 $2^5=32$

Cache地址格式



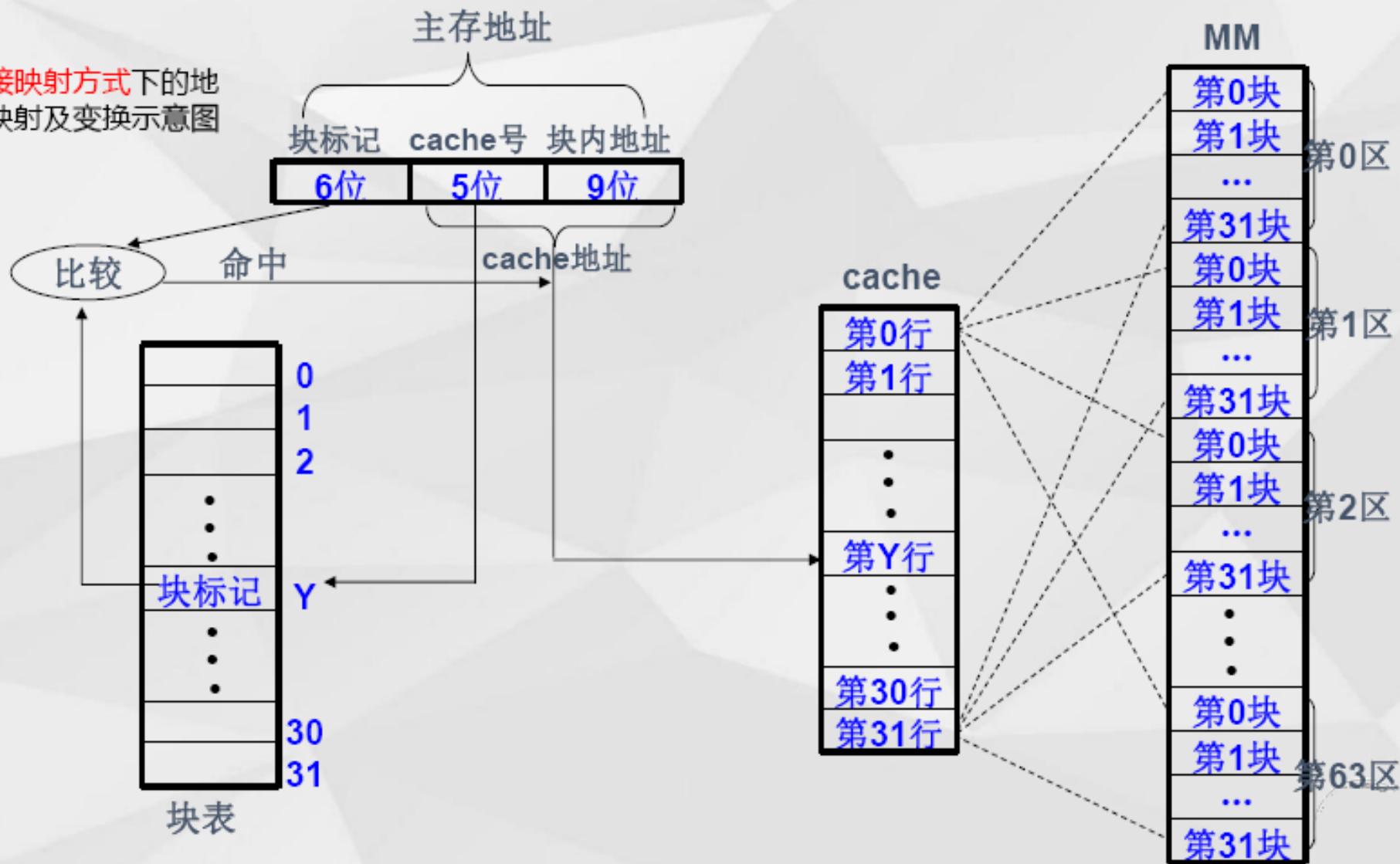
- 主存容量**1MB**，所以**主存地址**共**20位**

主存地址格式

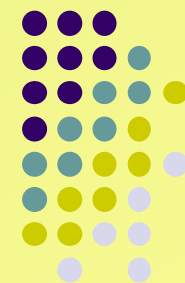


- 所以块表容量为 $2^5 \times 6$ 位

直接映射方式下的地址映射及变换示意图

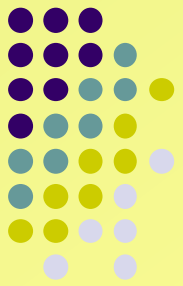


思考两种映射方式的优缺点



- 全相联检索成本高
- 直接映射换出换入机制有缺陷
- 能否折中？

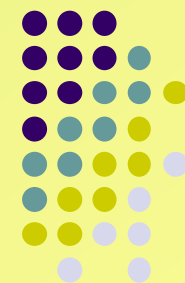
三、组相联映射方式



➤ 前两者的组合

- › Cache分组，组间采用直接映射方式，组内采用全相联的映射方式
- › Cache分组U，组内容量V，即U组V行

三、组相联映射方式



› 映射方法（一对多）

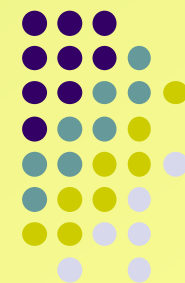
- ◆ $Q = j \bmod u$

- ◆ 主存第j块内容拷贝到Cache的q组中的某行

› 地址变换

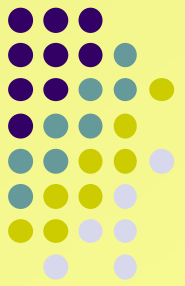
- ◆ 设主存内块地址x，看是不是在Cache中，先计算组号 $y = x \bmod u$ ，然后在y组中查找

三、组相联映射方式

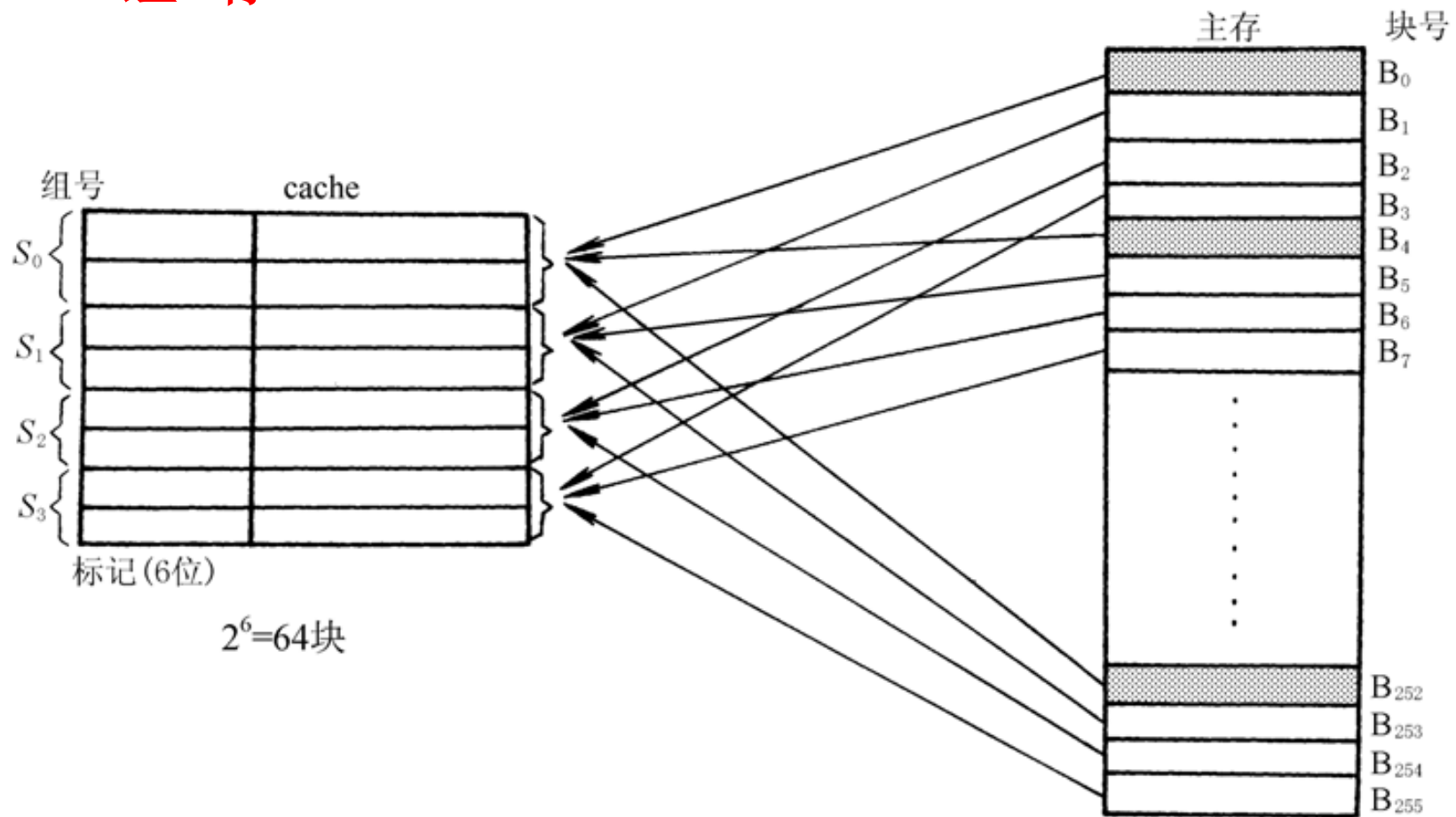


- › 分析：比全相联容易实现，冲突低
- › $v=1$ ，则为直接相联映射方式
- › $u=1$ ，则为全相联映射方式
- › v 的取值一般比较小，一般是2的幂，称之为 **v 路组相联Cache**。

三、组相联映射方式



U组V行



三、组相联映射方式

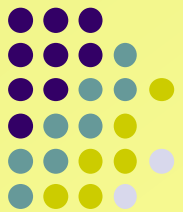
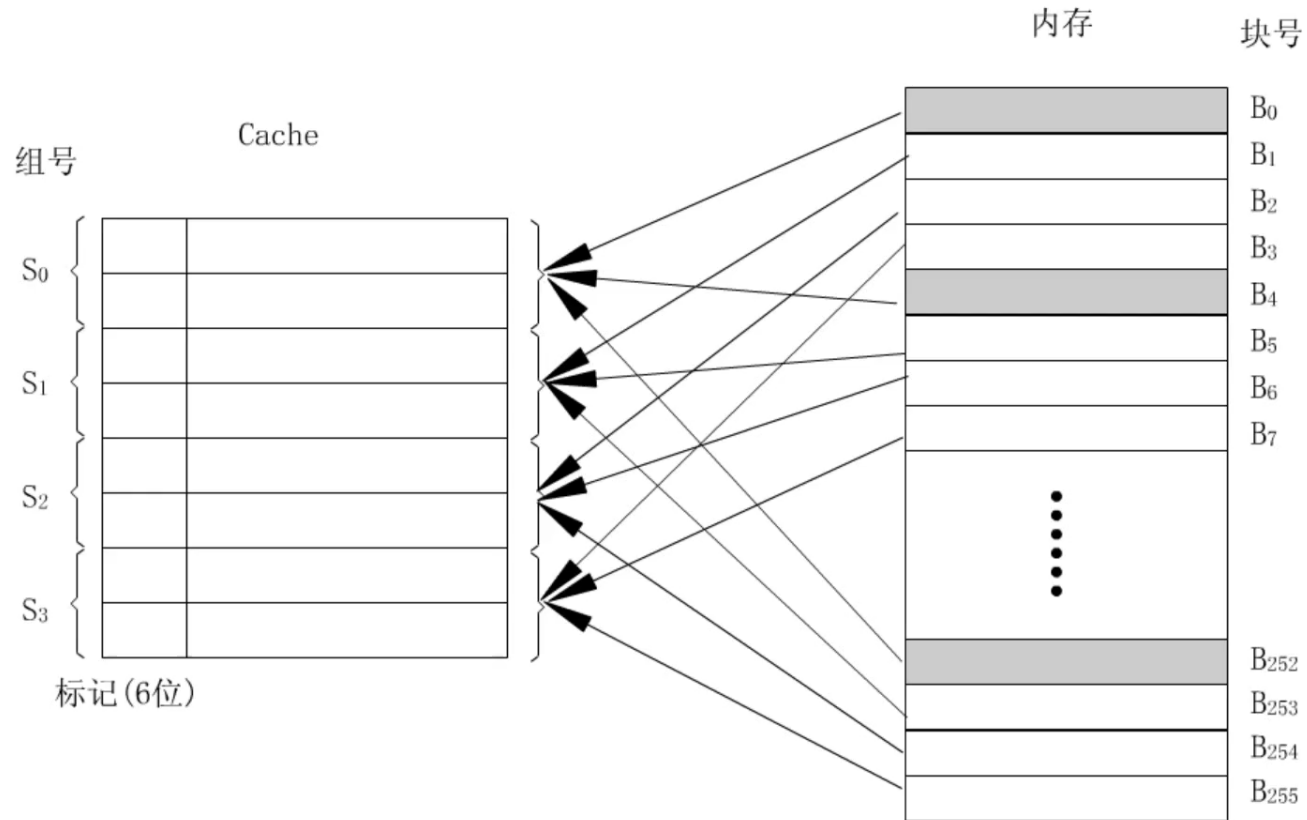


图3.35 组相联映射的Cache组织



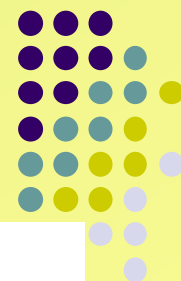
A decorative graphic consisting of a grid of colored dots in shades of purple, teal, yellow, and light blue, arranged in a pattern that tapers to the right.

蓝色：组号；绿色：字地址



(b) 组相联 cache 的检索过程

教材P107



[例] 直接映射方式的内存地址格式如下所示:

标记 s-r	行 r	字 w
8 位	14 位	2 位

若主存地址用十六进制表示为 BBBBBB, 请用十六进制格式表示直接映射方式 cache 的标记、行、字的值。

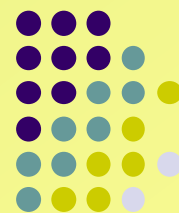
解 $(BBBBBB)_{16} = (1011\ 1011\ \underbrace{1011\ 1011\ 1011\ 1011}_{\text{行号}})_2$

标记 $s-r = (1011\ 1011)_2 = (BB)_{16}$

行 $r = (1011\ 1011\ 1011\ 10)_2 = (2EEE)_{16}$

字 $w = (11)_2 = (3)_{16}$

教材P107



[例] 一个组相联 cache 由 64 个行组成，每组 4 行。主存包含 4K 个块，每块 128 字。请表示内存地址的格式。

典型例题

解 块大小=行大小= 2^w 个字=128= 2^7 $\therefore w=7$

每组的行数 $k=4$

Cache 的行数= $kv=k \times 2^d = 4 \times 2^d=64$ $\therefore d=4$

组数 $v=2^d=2^4=16$

主存的块数= $2^s=4K=2^2 \times 2^{10}=2^{12}$ $\therefore s=12$

标记的大小 $(s-d)$ 位= $12-4=8$ 位

主存地址长度 $(s+w) = 12+7=19$ 位

主存寻址单元数 $2^{s+w}=2^{19}$

故 $k=4$ 合各组相联的内存地址格式如下所示：

标记 s-d	组号 d	字号 w
8 位	4 位	7 位

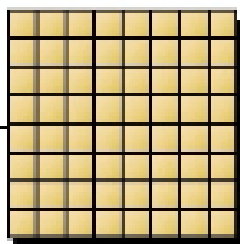
Cache读操作

命中 (HIT)

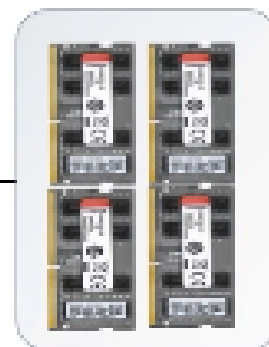


字

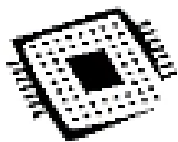
Cache



块

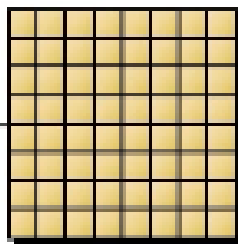


缺失 (MISS)

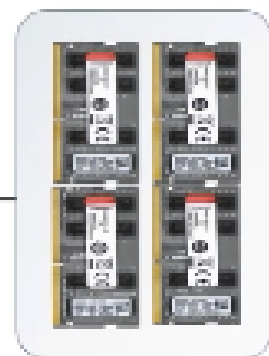


字

Cache

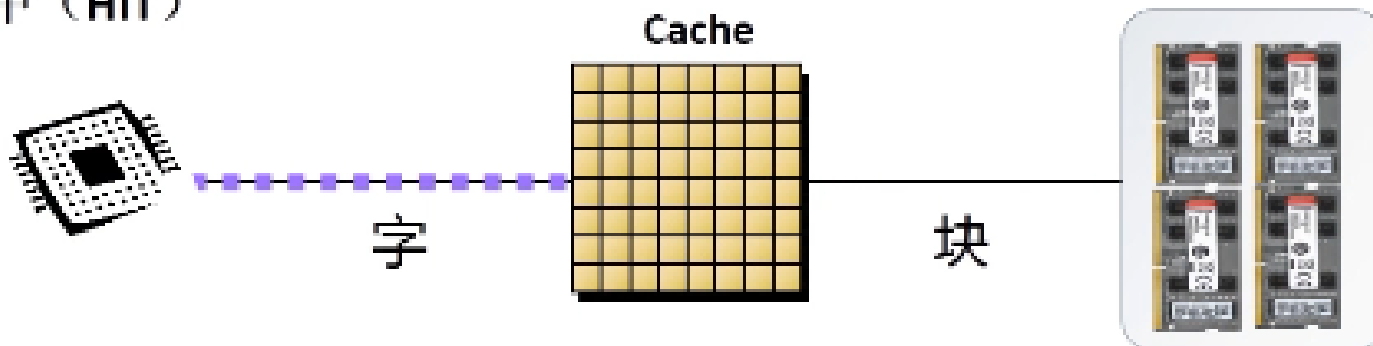


块

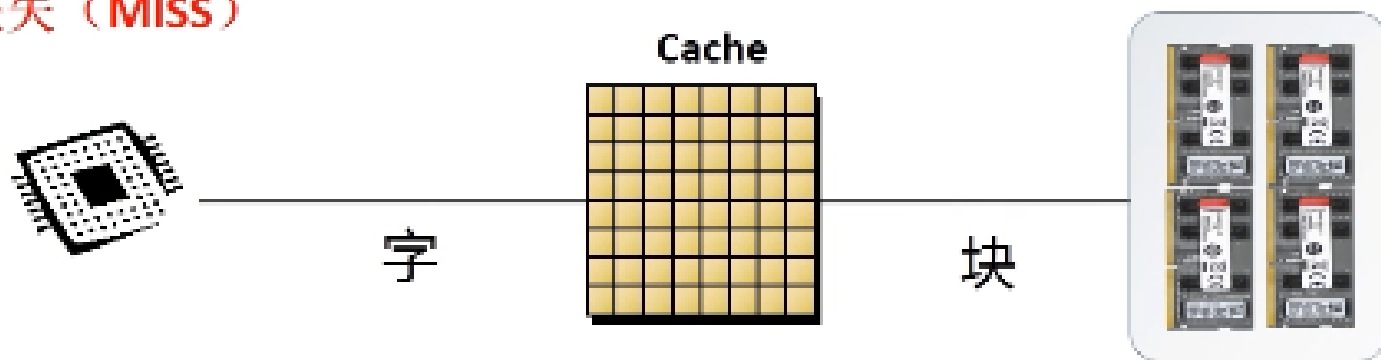


Cache读操作

命中 (HIT)

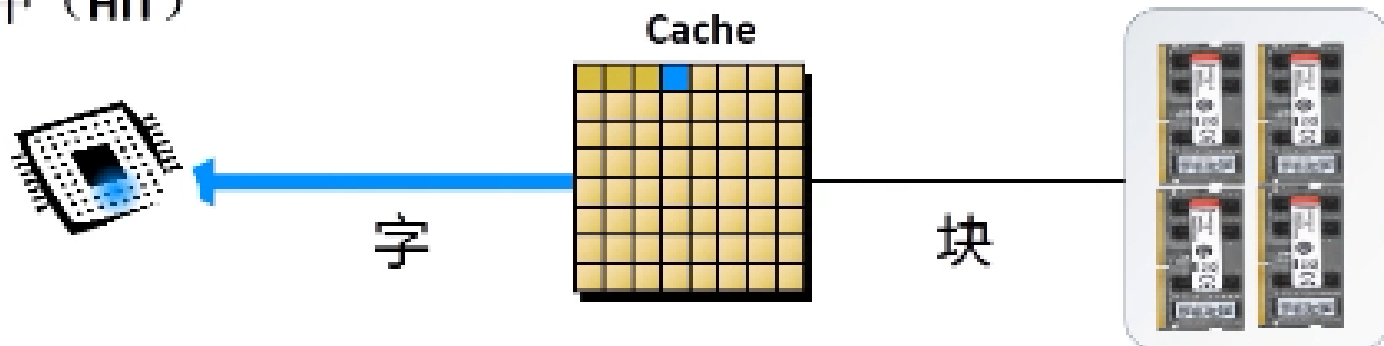


缺失 (MISS)

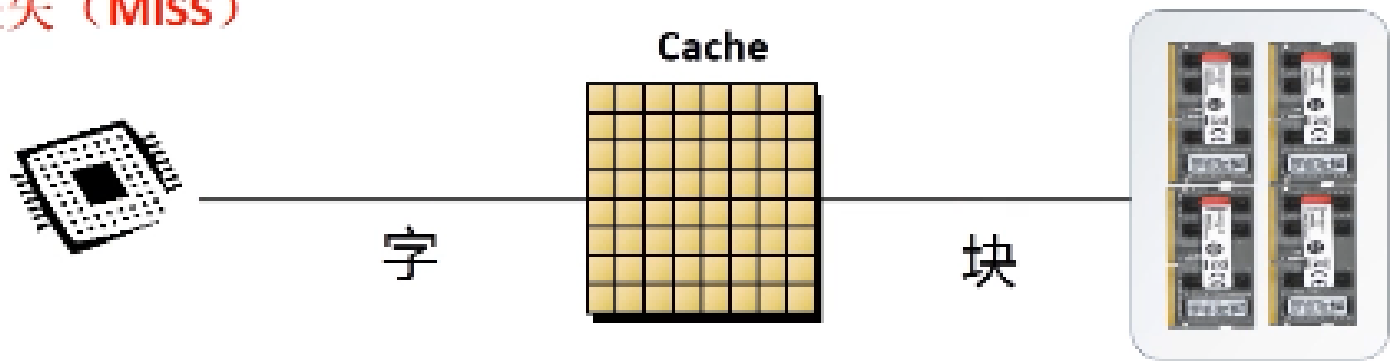


Cache读操作

命中 (HIT)



缺失 (MISS)



①如何判断数据在Cache中？

②Cache中的数据有效吗？

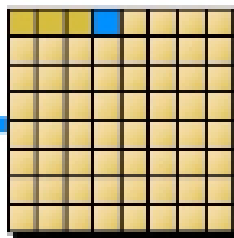
Cache读操作

命中 (HIT)

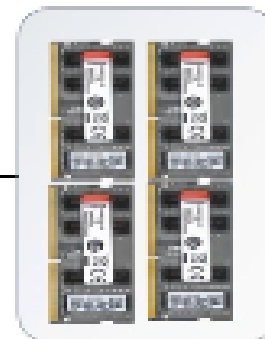


字

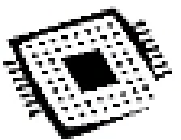
Cache



块

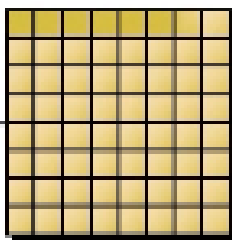


缺失 (MISS)

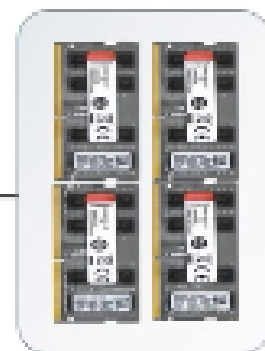


字

Cache

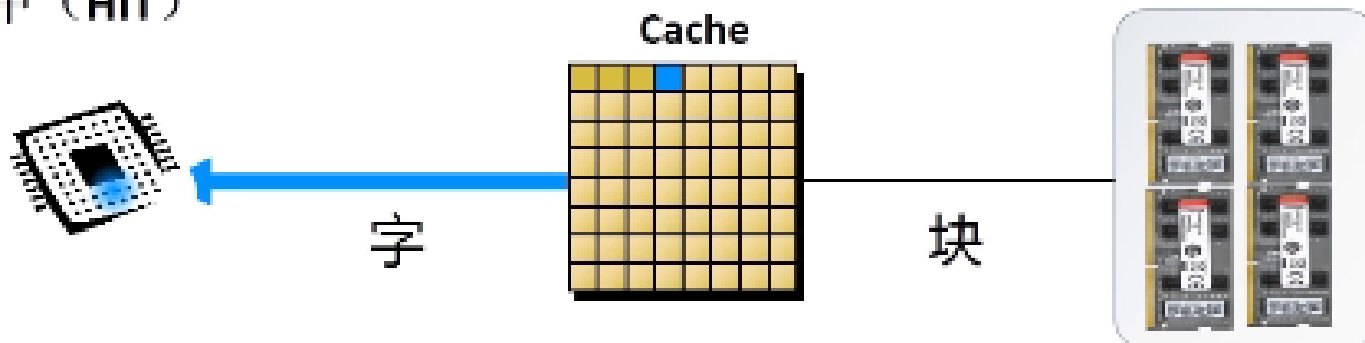


块

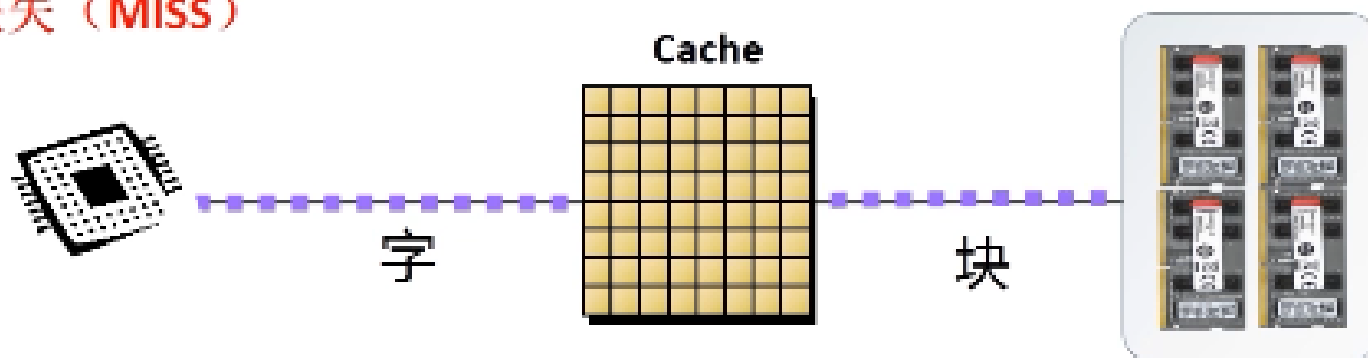


Cache读操作

命中 (HIT)

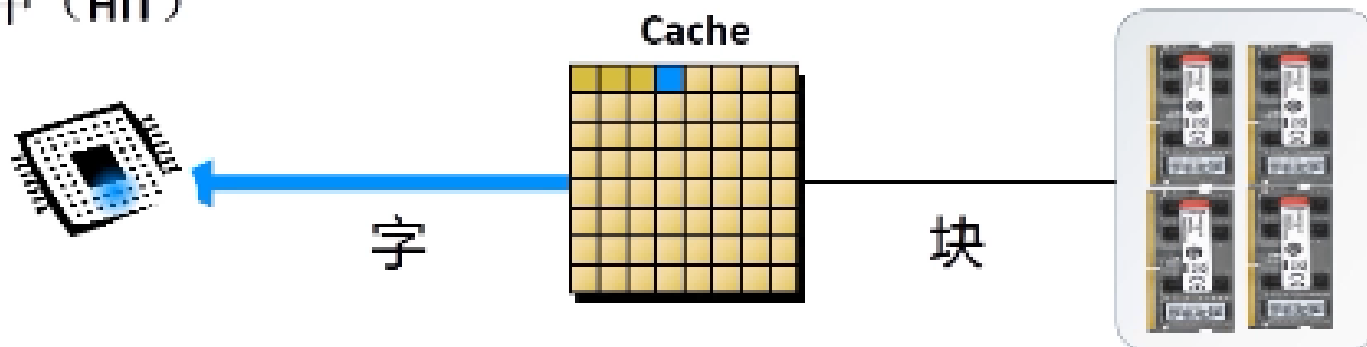


缺失 (MISS)

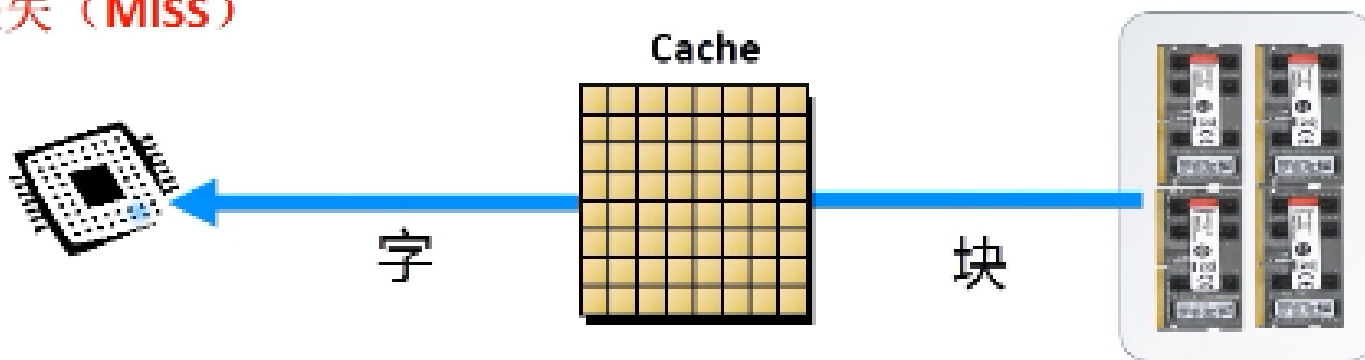


Cache读操作

命中 (HIT)

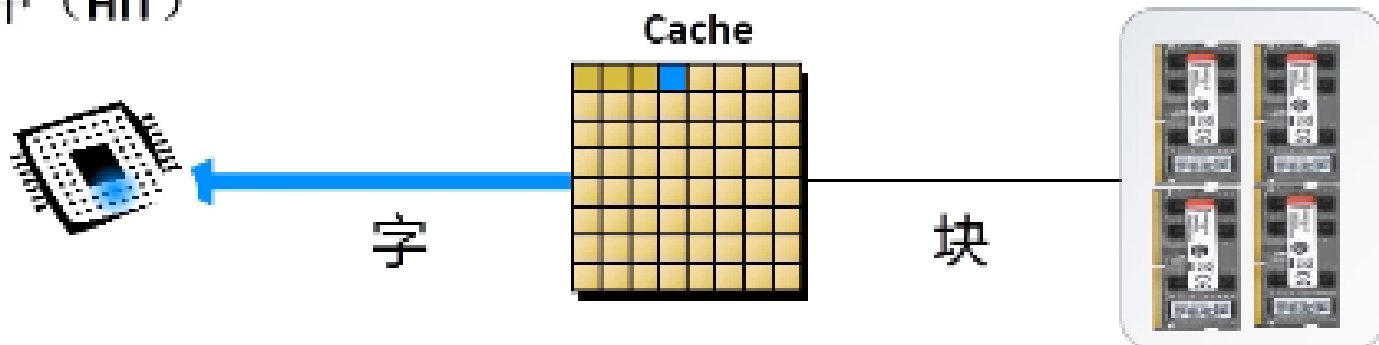


缺失 (MISS)

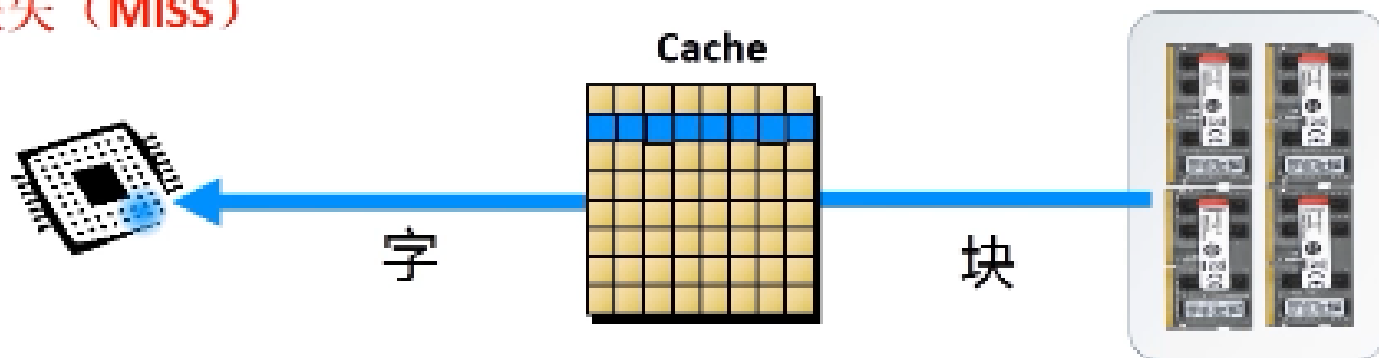


Cache读操作

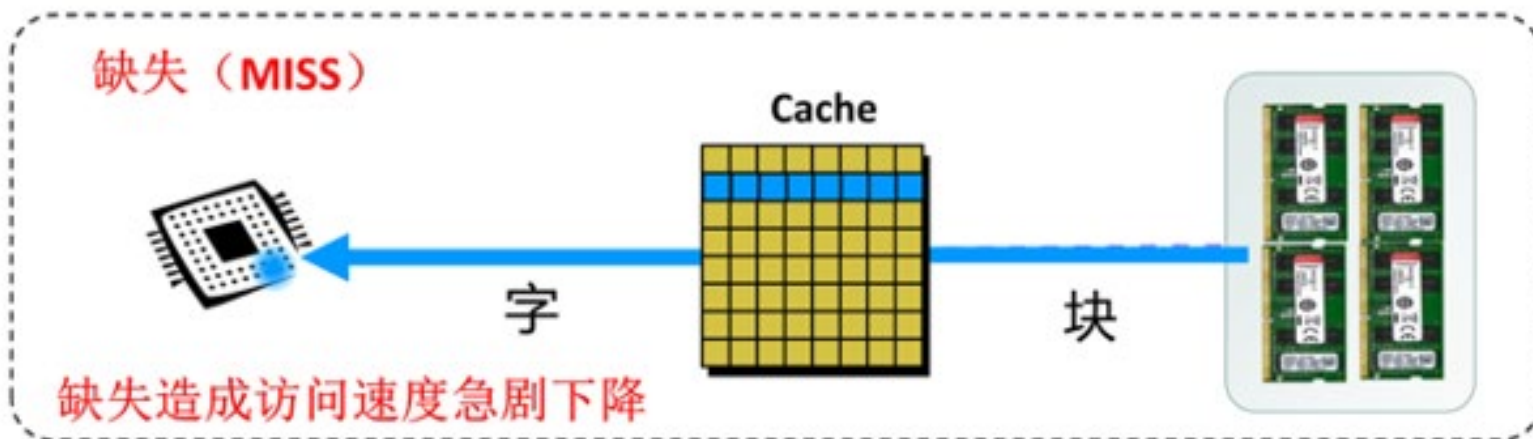
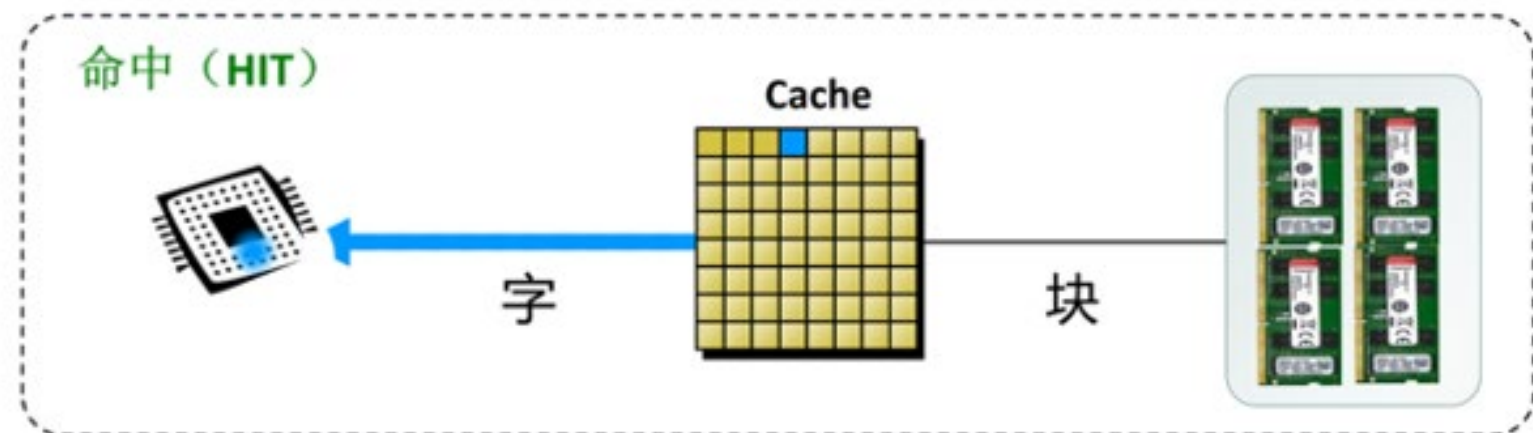
命中 (HIT)



缺失 (MISS)



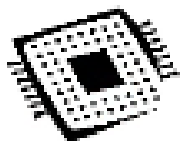
Cache读操作



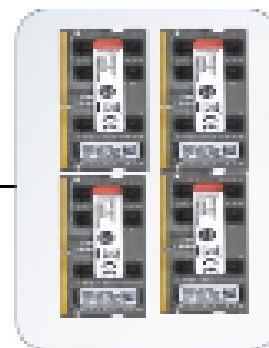
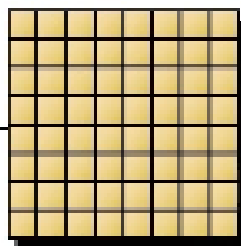
Cache写操作

全写法

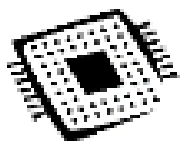
写穿策略 WriteThrough



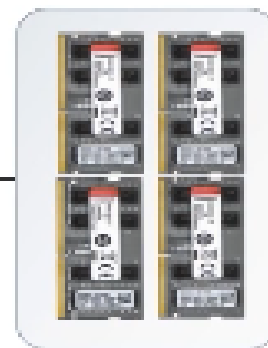
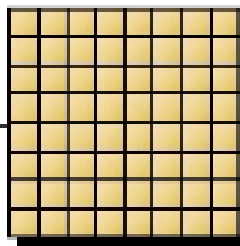
Cache



写回策略 Write Back

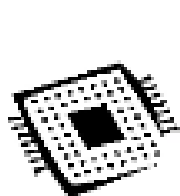


Cache

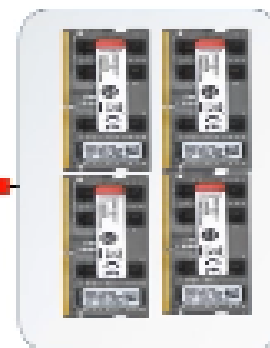
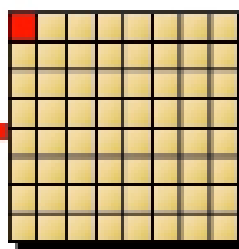


Cache写操作

写穿策略 WriteThrough



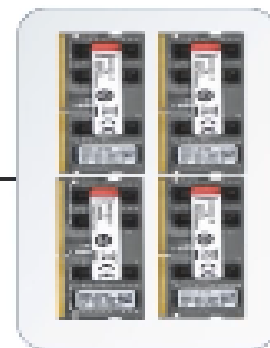
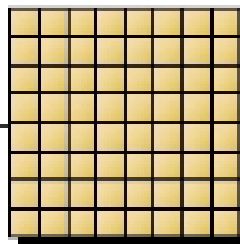
Cache



写回策略 Write Back

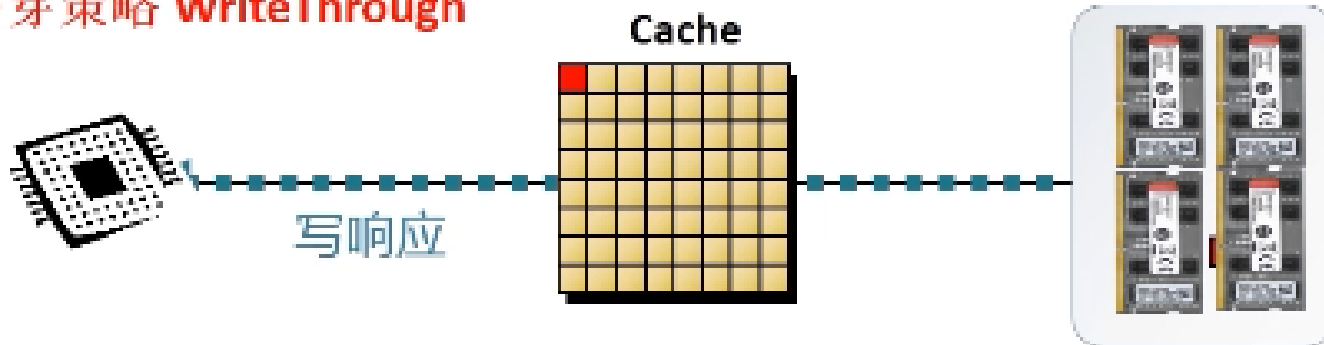


Cache

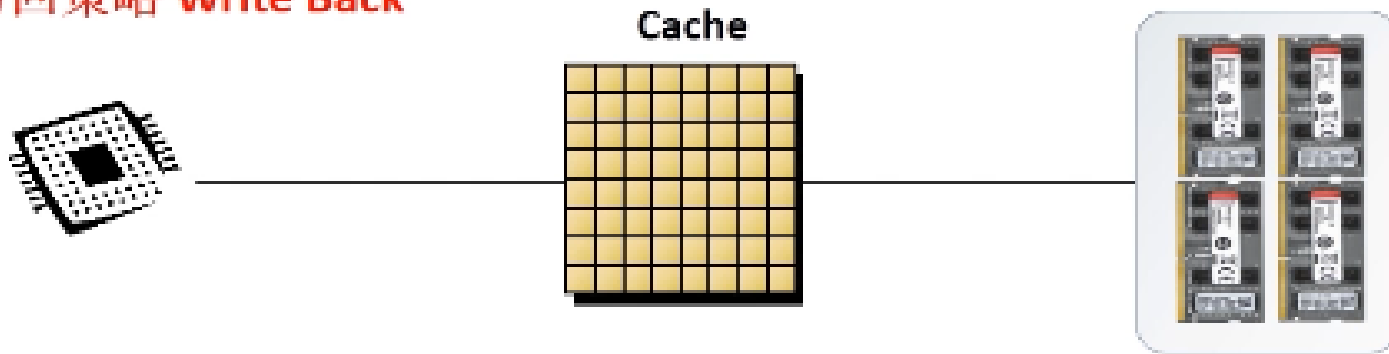


Cache写操作

写穿策略 WriteThrough

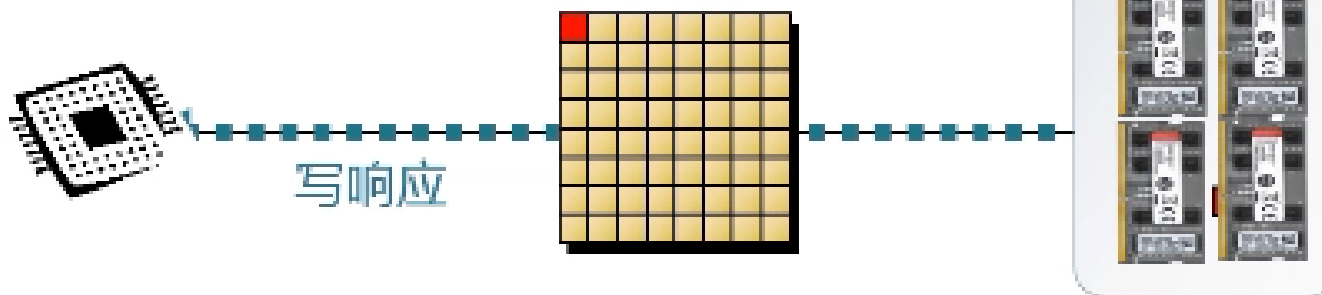


写回策略 Write Back

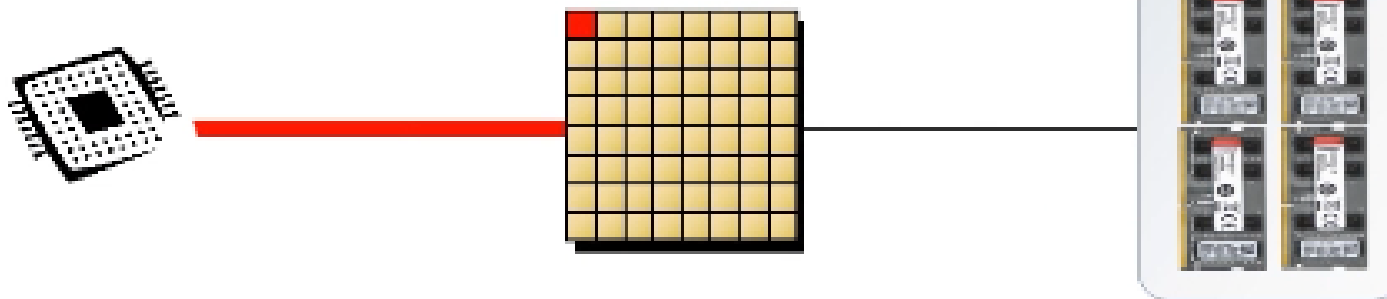


Cache写操作

写穿策略 WriteThrough

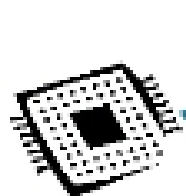


写回策略 Write Back



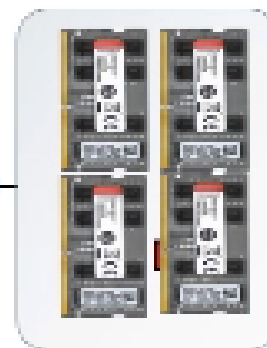
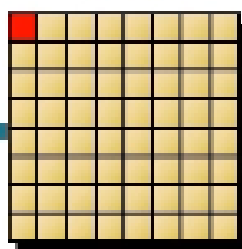
Cache写操作

写穿策略 WriteThrough

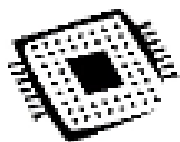


写响应

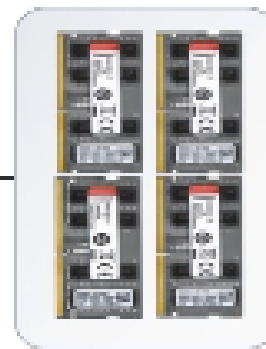
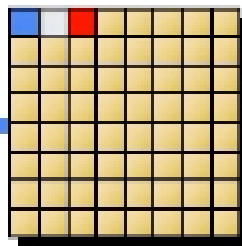
Cache



写回策略 Write Back

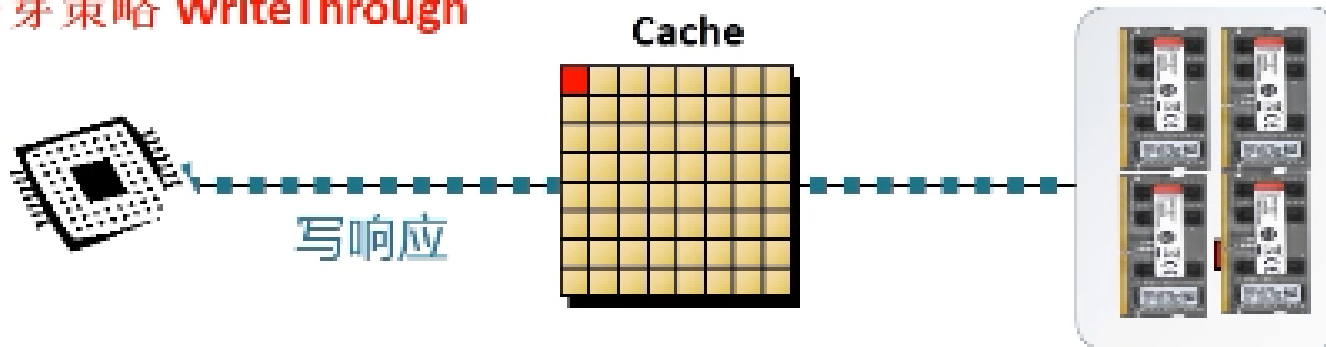


Cache

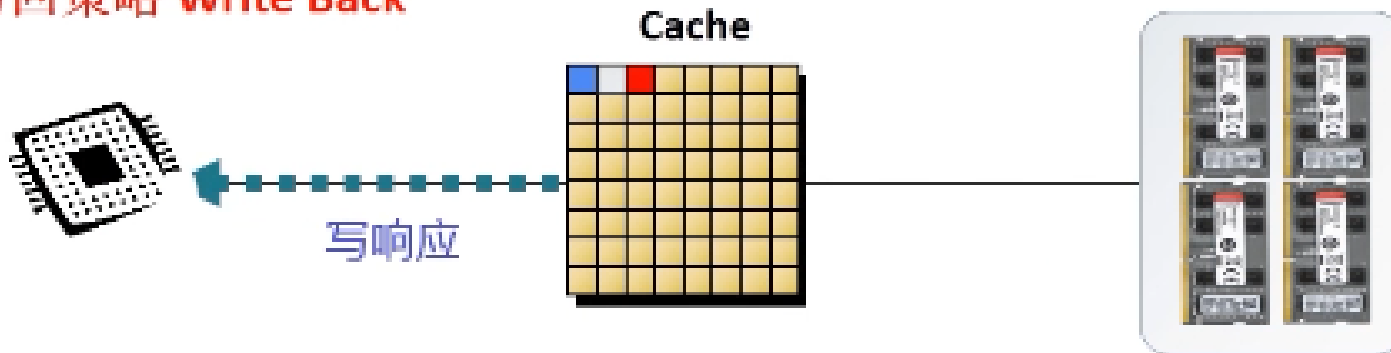


Cache写操作

写穿策略 WriteThrough



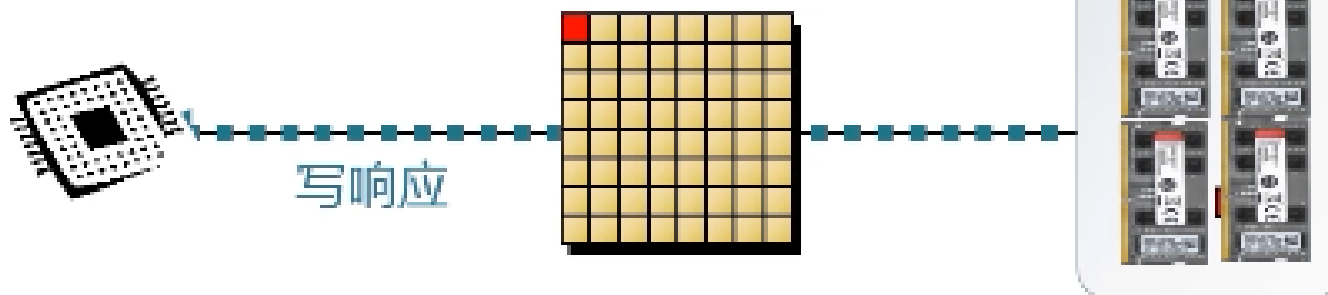
写回策略 Write Back



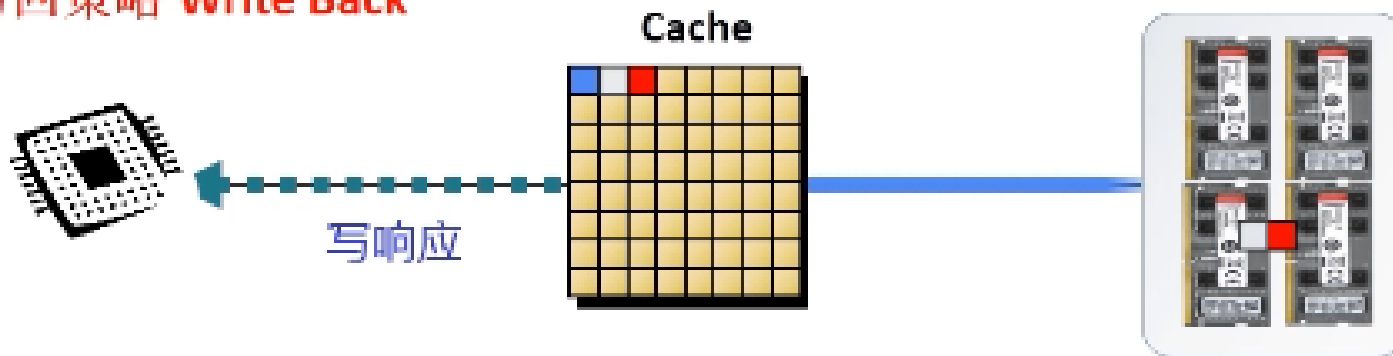
替换时，根据Dirty位，若为1，Cache行写入主存，保证一致性

Cache写操作

写穿策略 WriteThrough

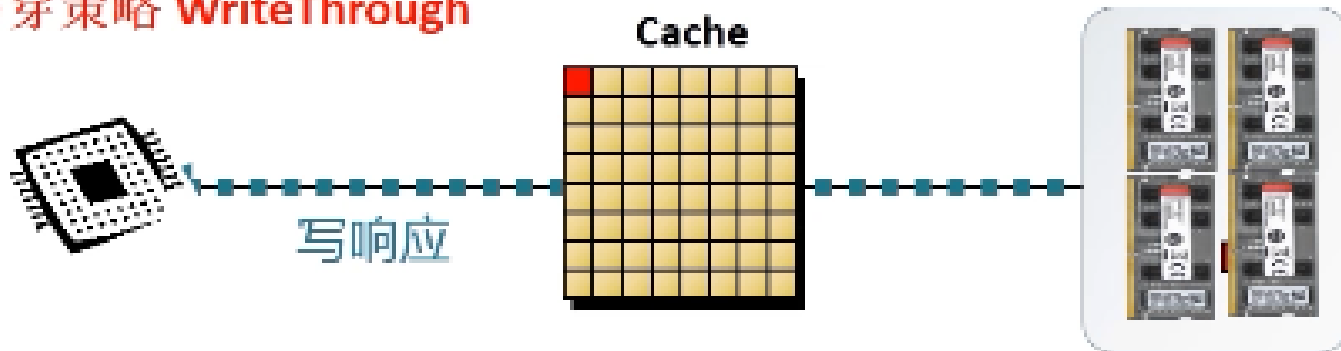


写回策略 Write Back

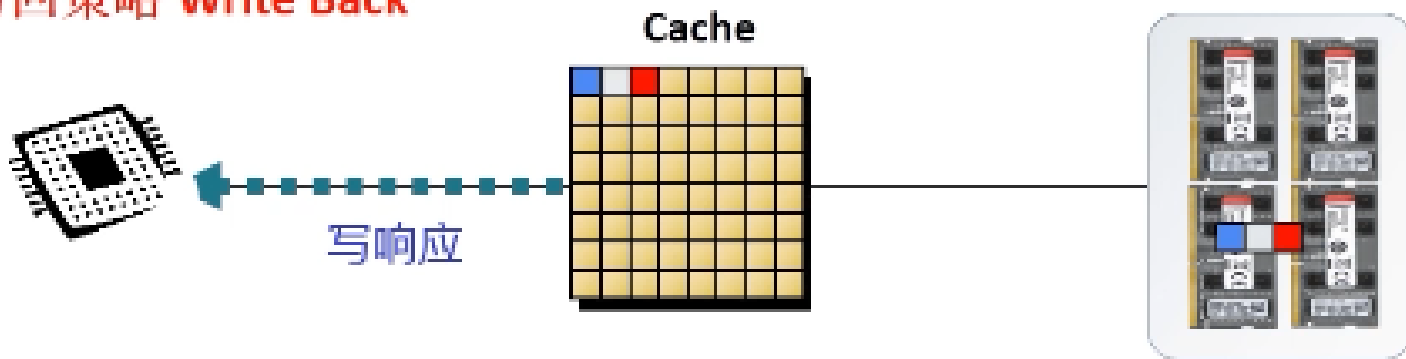


Cache写操作

写穿策略 WriteThrough

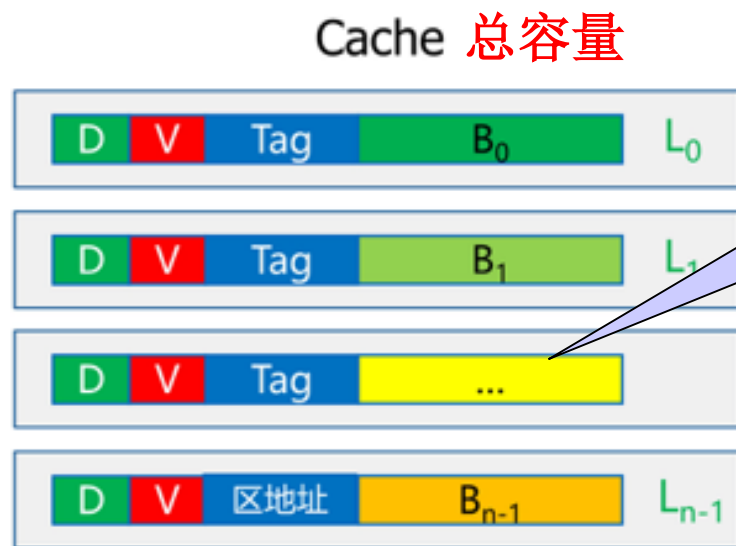


写回策略 Write Back



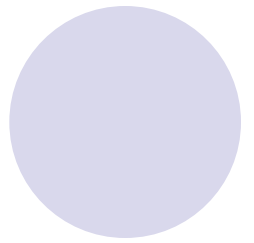
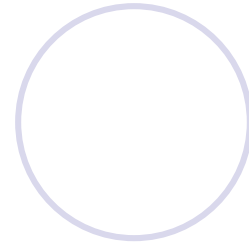
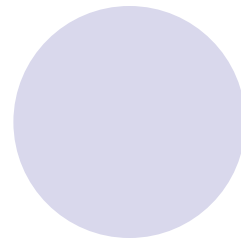
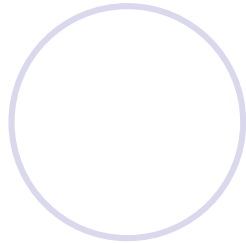
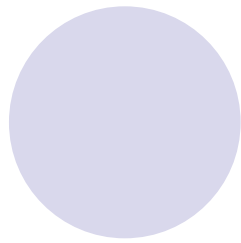
3.6.3 替换策略

需要替换算法的原因



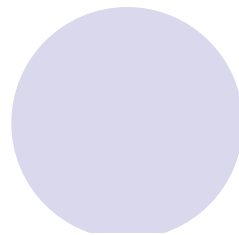
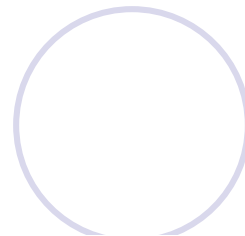
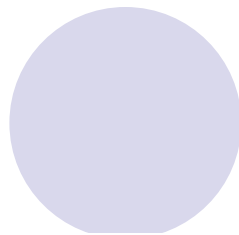
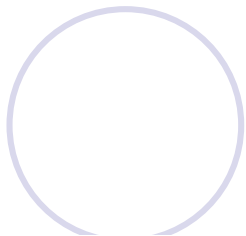
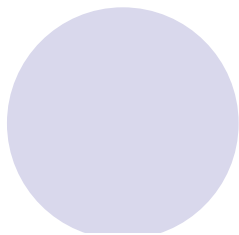
通常讲的**Cache容量**指的是**Cache数据区**

程序运行一段时间后，Cache存储空间被占满，当再有新数据要调入时，就需要通过某种机制决定替换的对象。

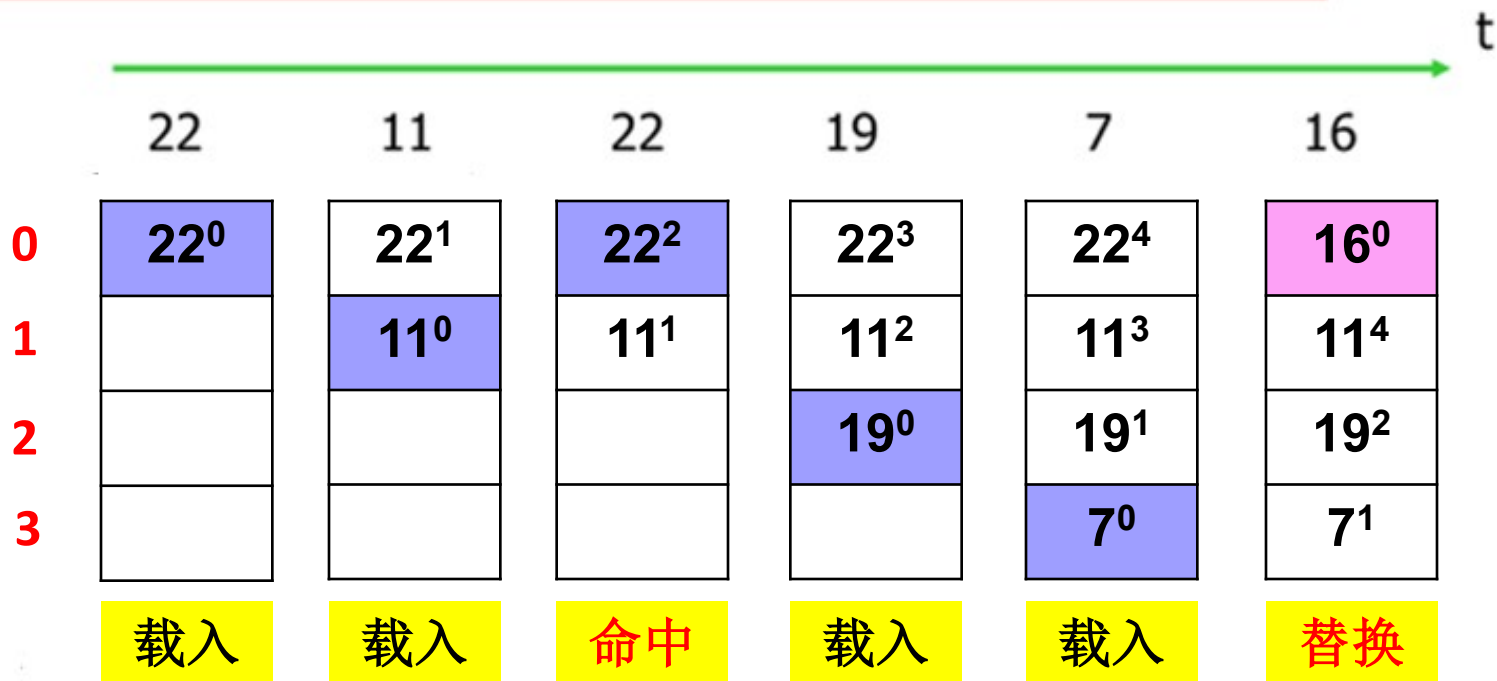


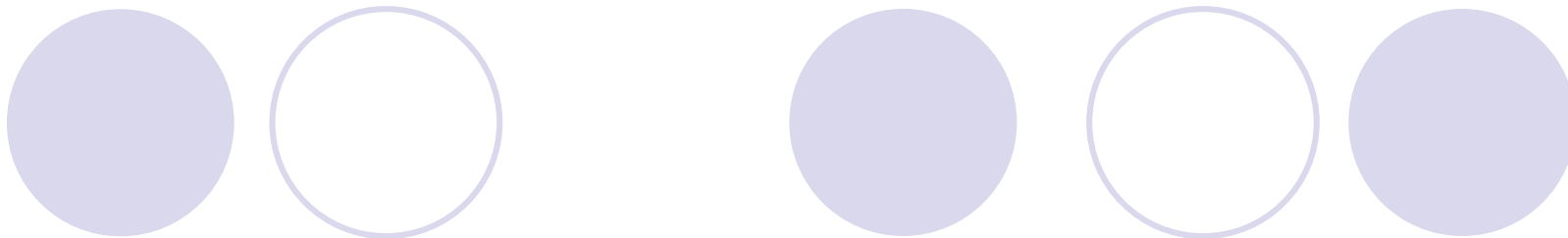
几种常见的替换算法

- 先进先出法-FIFO (First in First out)
- 最不经常使用法---LFU (Least Frequently Used)
- 近期最少使用法--- LRU (Least recently used)
- 随机替换法



替换算法FIFO



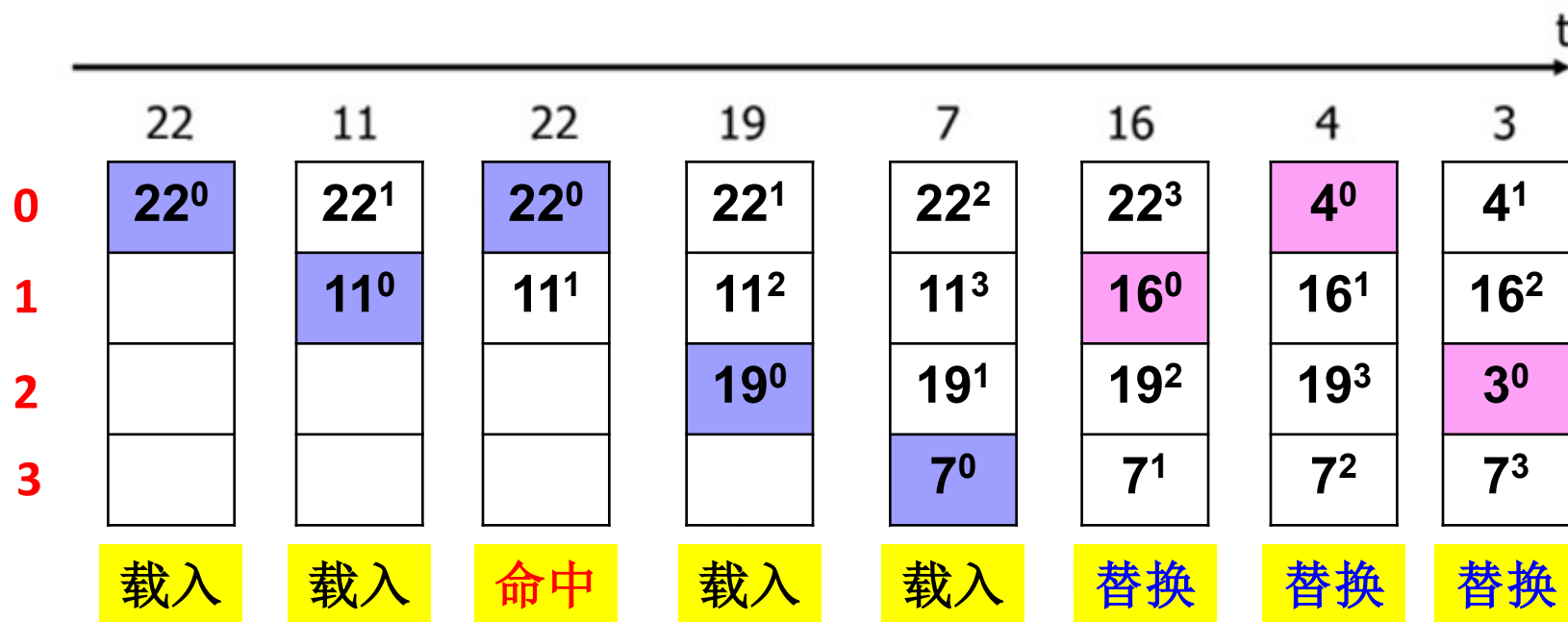


替换算法LFU

	t								
	22	11	22	19	11	16	22	4	3
0	22 ⁰	22 ⁰	22 ¹	22 ¹	22 ¹	22 ¹	22 ²	22 ²	22 ²
1		11 ⁰	11 ⁰	11 ⁰	11 ¹	11 ¹	11 ¹	11 ¹	11 ¹
2				19 ⁰	19 ⁰	19 ⁰	19 ⁰	4 ⁰	4 ⁰
3						16 ⁰	16 ⁰	16 ⁰	3 ⁰
	载入	载入	命中	载入	命中	载入	命中	替换	替换

重点掌握

替换算法LRU



使用多级cache减少缺失损失

华为公司设计的鲲鹏 920 处理器是一种多核CPU，在其单核CPU上采用了一种片上**两级Cache**的架构，即通过一层容量较大的**L2-Cache**来处理容量较小的**L1-Cache**的缺失(miss)，从而降低整体的缺失代价，其基本思想可以按照如下设计：

- a) L1-Cache 的块总是存在于L2-Cache中
- b) L1中miss, 但在L2中命中，则从L2拷贝相应的块到L1
- c) 在L1和L2中均miss, 则从共享主存拷贝相应的块到L1和L2
- d) 对L1写操作导致将数据同时写到L1和L2

假设在单个处理器内核上采用了该两级Cache架构，其中L1-Cache的存取需要1个时钟周期，L2-Cache的存取需要8个时钟周期。如果发生了Cache缺失那么访问共享主存需要耗费18个时钟周期；而L1-Cache发生缺失的概率是L2-Cache的两倍。如果已知该单核处理器的存储系统平均访问时间是2个时钟周期，请分别**计算L1和L2的缺失率**。

2021年期末考试题

答：对于多级Cache设计，其平均访问时间可以由下公式表示：

$$T_{avg} = H_1 * C_1 + (1 - H_1) * (H_2 * C_2 + (1 - H_2) * M)$$

H_1 是 L_1 缓存的命中率， H_2 是 L_2 缓存的命中率，

C_1 是获取 L_1 缓存中数据需要的时间， C_2 是 L_1 缺失以后的读取时间， M 是 L_2 缺失以后的读取时间

由本题的 $T_{avg} = 2 \text{ Cycle}$ ， $C_1 = 1 \text{ Cycle}$ ， $C_2 = 8 \text{ Cycle}$ ， $M = 18 \text{ Cycle}$ 。
而且 $1 - H_1 = 2(1 - H_2)$

代入公式： $2 = H_1 + (1 - H_1) * (H_2 * 8 + (1 - H_2) * 18)$

求解得：

$$H_1 = 0.869338, H_2 = 0.934669$$

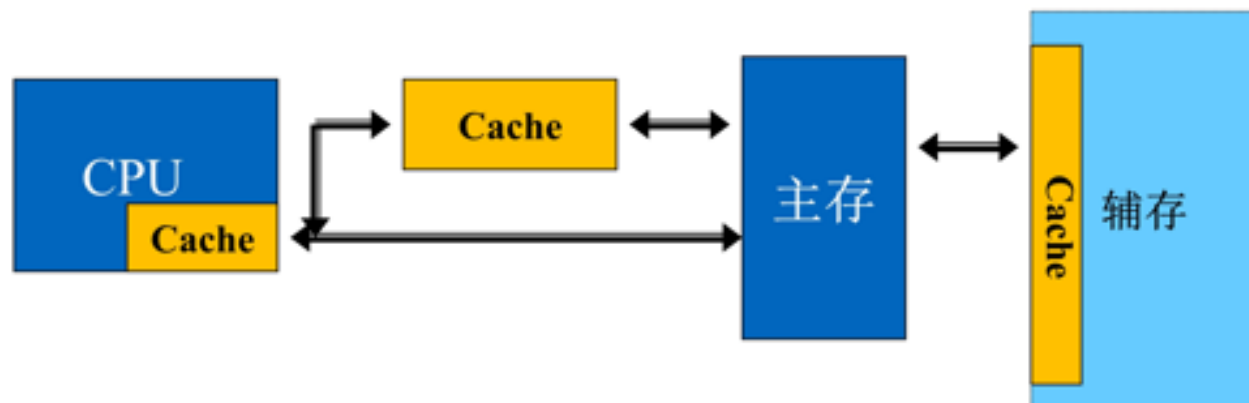
因此 L_1 的缺失率为 $1 - H_1 = 0.130662$

L_2 的缺失率为 $1 - H_2 = 0.065331$

https://blog.csdn.net/weixin_30861459/article/details/96263521

3.7 虚拟存储器

虚拟存储器概述



- 冯诺依曼计算机工作原理：存储程序、程序控制
- 计算机能执行比主存空间大的程序吗？
- 1961年英国曼彻斯特大学Kilbrn等人提出, 70年代广泛地应用于大中型计算机系统中。

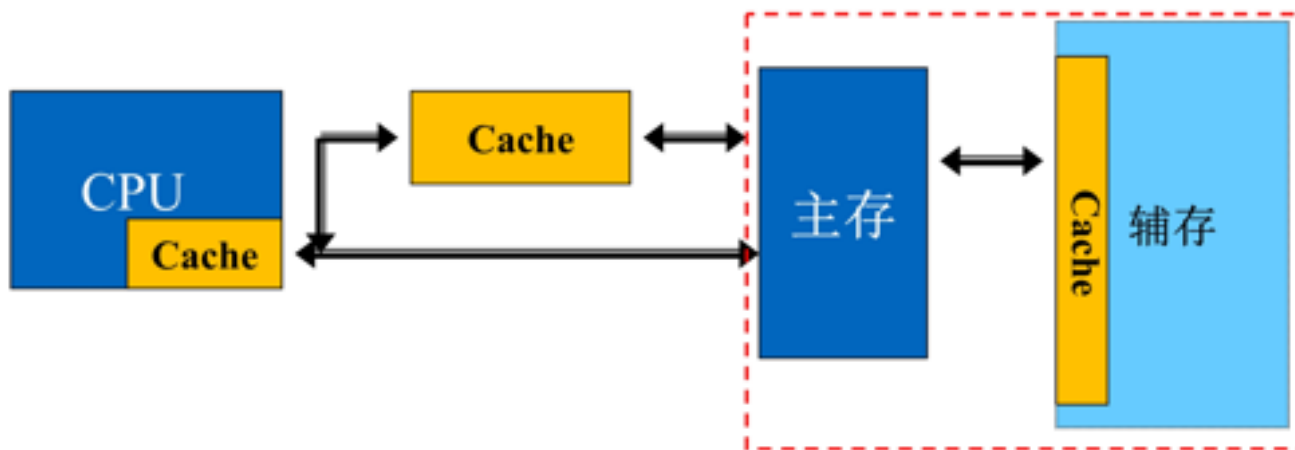
3.7 虚拟存储器

- 说明：

这部分内容将在《操作系统》课程中用完整一章详细介绍，本课仅简单引入。

3.7 虚拟存储器

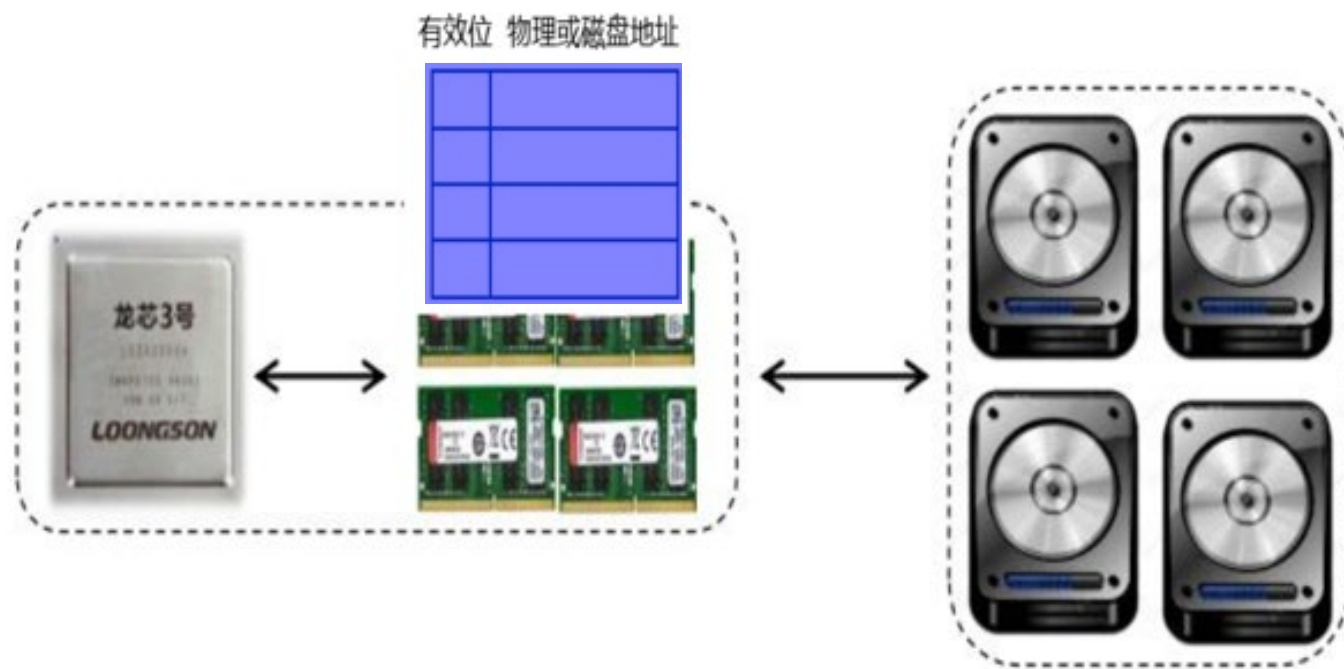
虚拟存储器概述



- 处于主存 - 辅存存储层次
- 解决主存容量不足的问题，向程序设计者提供比主存空间大的编程空间
- 分类：页式虚拟存储器、段式虚拟存储器、段页式虚拟存储器

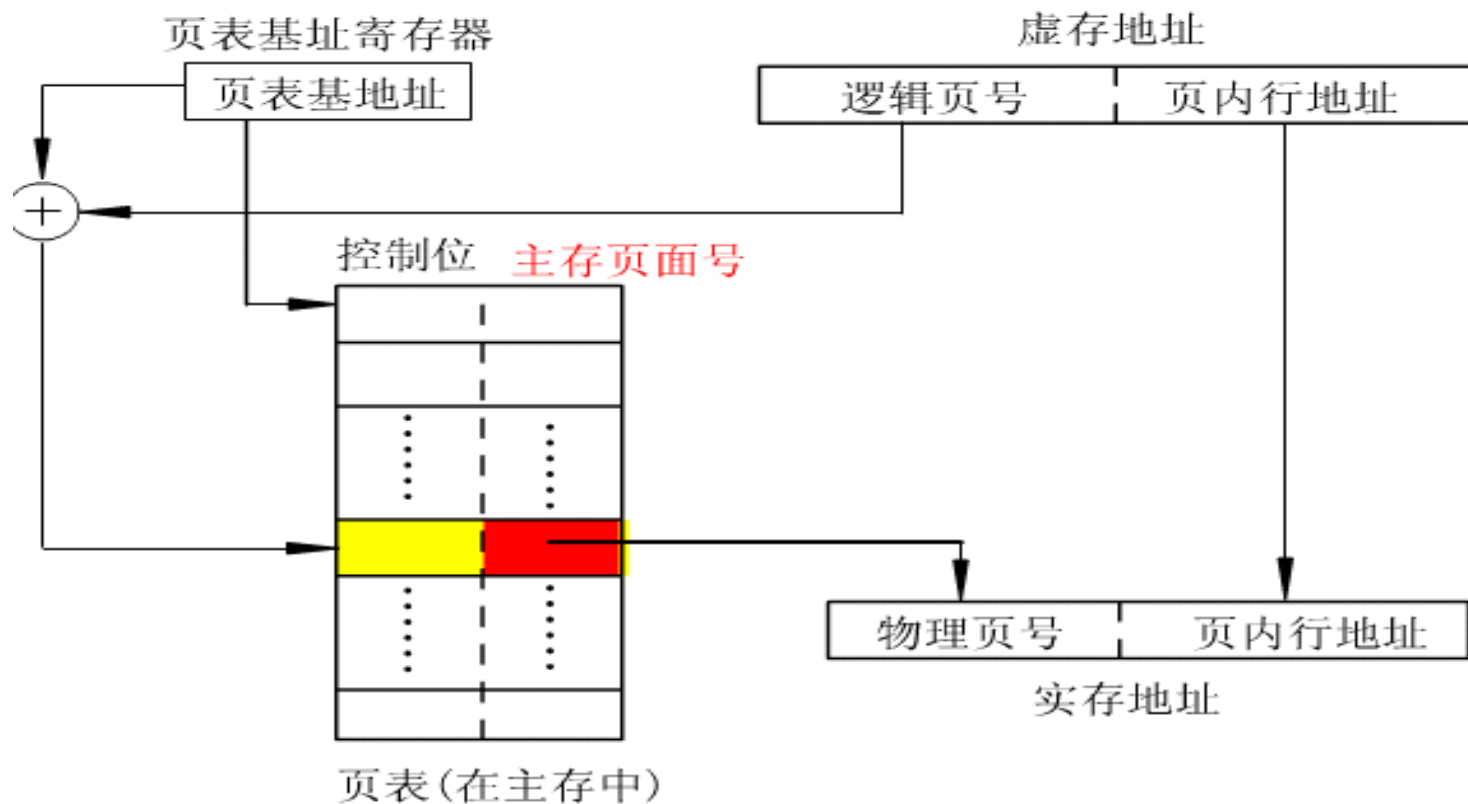
3.7 虚拟存储器

虚拟存储器技术必须解决的问题



逻辑地址 $\longrightarrow ? \longrightarrow$ 物理地址

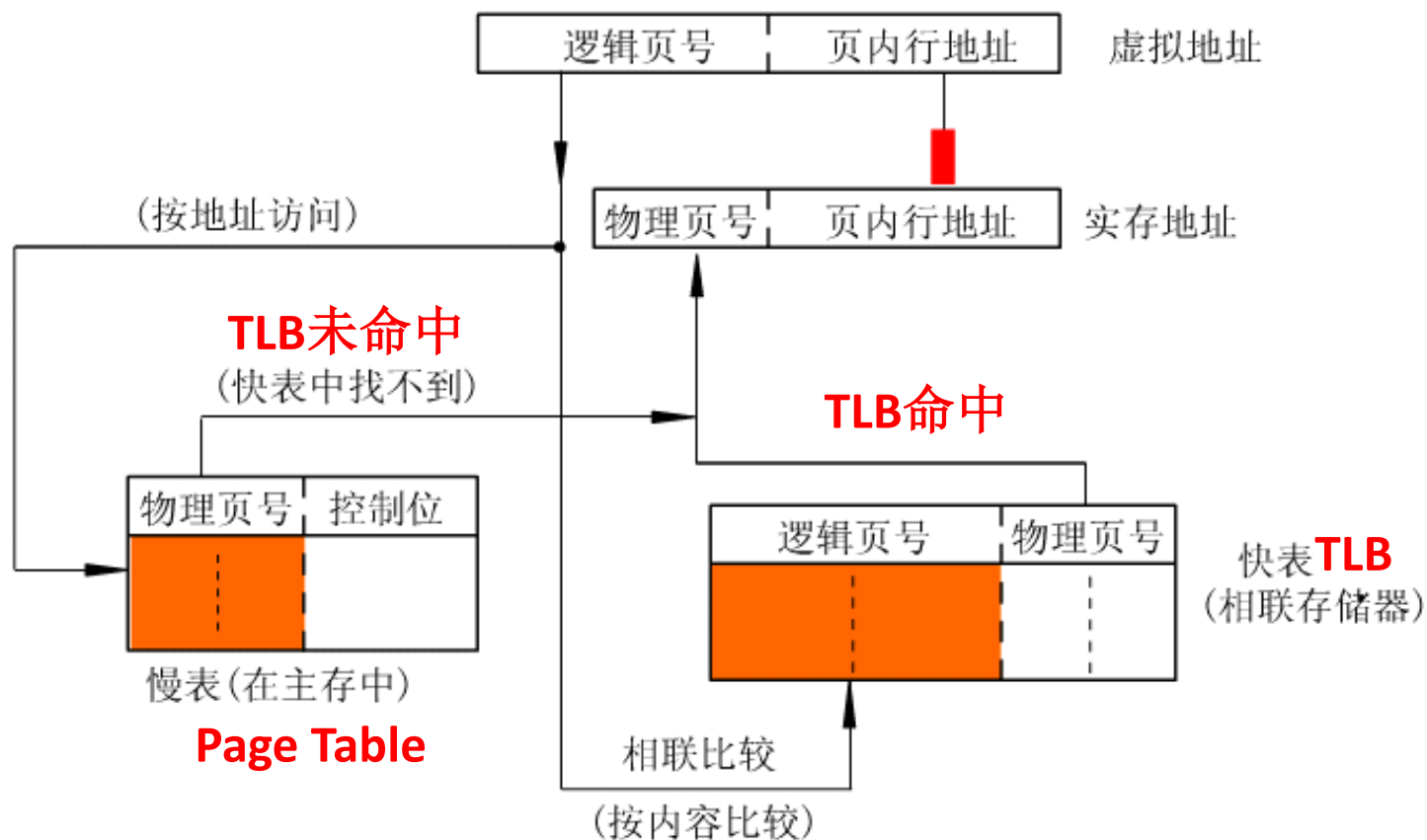
1. 页式管理的地址变换演示



在页表中, 对应**每一个**虚存逻辑页号有一个**目录**, 它至少包含**主存页面地址**.

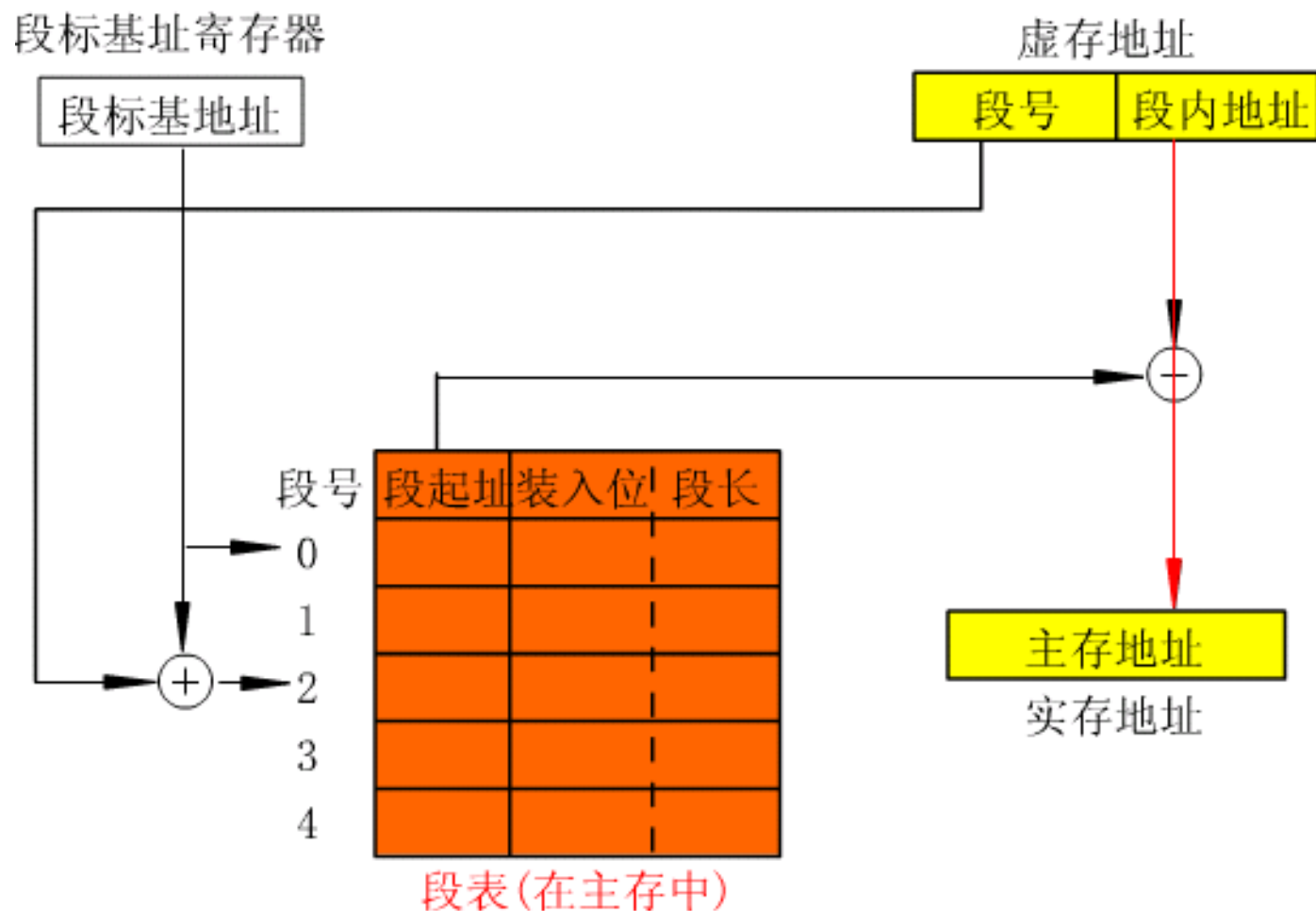
快表与慢表实现内部地址变换的方式

- 避免页表已保存或已调入主存储器时对主存访问次数的增多, 把页表的最活跃部分存放在高速存储器中组成快表。

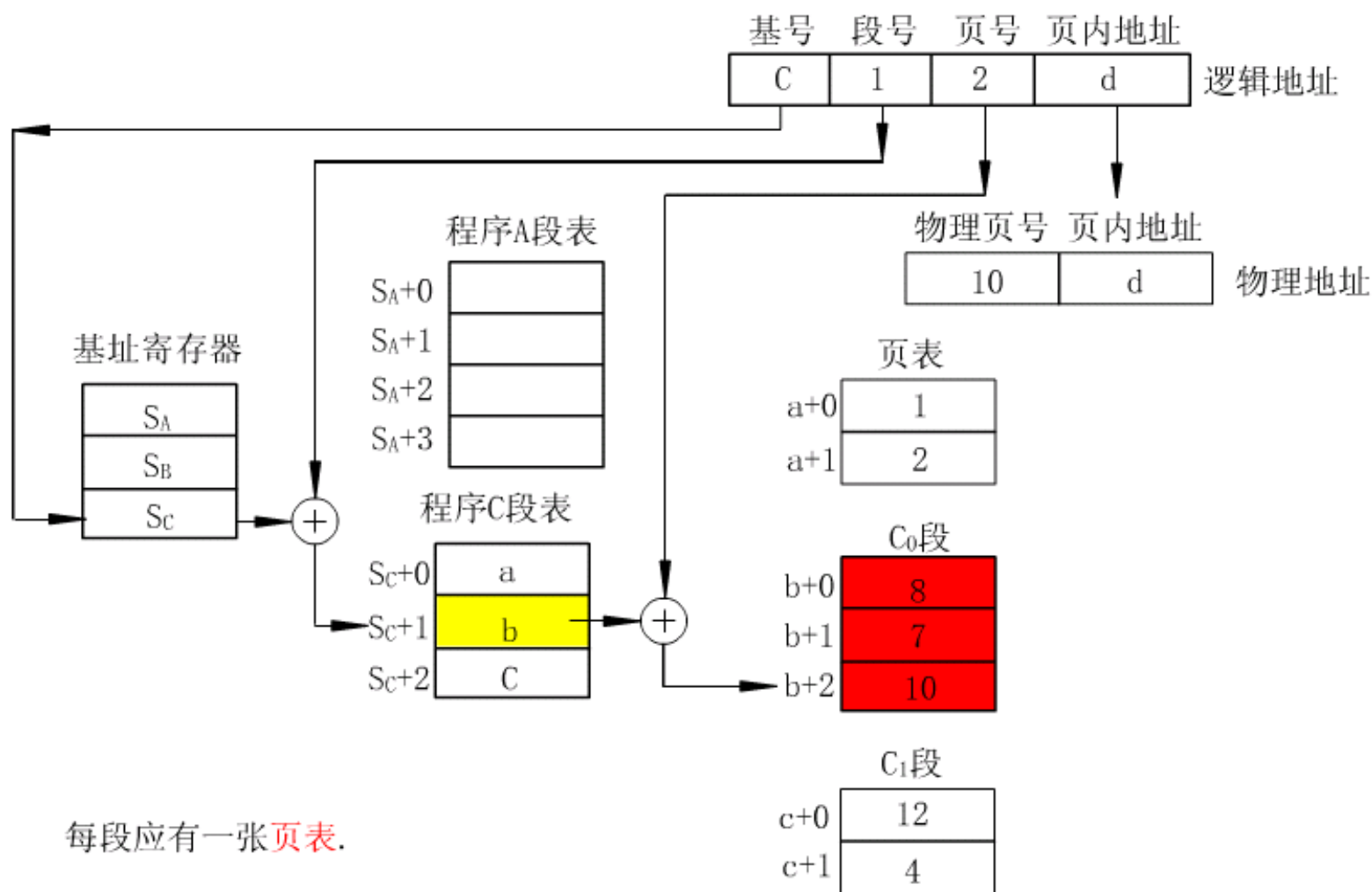


TLB: Translation Lookaside Buffer, 地址转换后备缓冲器

2. 段式管理的地址变换演示



3. 段页式管理的地址变换演示



例题：

- 下图表示使用页表的虚实地址转换条件，页表存放在相联存储器中，其容量为8个存储单元，问：当CPU按虚地址1，2，3去访问主存时，主存的实地址码是多少？

虚拟地址	页号	页内地址
1	15	0324
2	7	0128
3	48	0516

页号	该页在主存中的起始地址
33	42000
25	38000
7	96000
6	60000
4	40000
15	80000
5	50000
30	70000

页表

解答：

- (1) 虚地址1，主存实地址码： $80000+0324=80324$
- (2) 虚地址2，主存实地址码： $96000+0128=96128$
- (3) 虚拟地址3的页号为48，当用48作检索项在页表中检索时，没有检索到页号为48的页面，此时操作系统暂停用户作业程序的执行，转去执行查页表程序。**如果该页面在主存中**，则将该页号及该页在主存中的起始地址写入页表；**如果该页面不在主存**，则操作系统要将该页面从外存调入主存，然后将页号及其在主存中的起始地址写入页表。

【例】假设主存只有a, b, c三个页框，组成a进c出的FIFO队列，进程访问页面的序列是0, 1, 2, 4, 2, 3, 0, 2, 1, 3, 2号。若采用①FIFO算法，②FIFO算法+LRU算法，用列表法分别求两种替换策略情况下的命中率。

【解】

页面访问序列		0	1	2	4	2	3	0	2	1	3	2	命中率
FIFO 算法	a	0	1	2	4	4	3	0	2	1	3	3	2/11=
	b		0	1	2	2	4	3	0	2	1	1	
	c			0	1	1	2	4	3	0	2	2	18.2%
						命中						命中	
FIFO+ LRU 算法	a	0	1	2	4	2	3	0	2	1	3	2	3/11=
	b		0	1	2	4	2	3	0	2	1	3	
	c			0	1	1	4	2	3	0	2	1	27.3%
						命中			命中			命中	

3.8 鲲鹏920处理器的内存存储系统

鲲鹏920处理器片上系统的内存存储系统

➤ 鲲鹏存储系统的层次结构

❑ 多级Cache：片内共享L3 Cache架构

⊗ 每个内核私有的L1 Cache和L2 Cache

❑ 64KB L1 I-Cache

❑ 64KB L1 D-Cache

❑ 512KB L2 Cache

⊗ 每个超级内核集群配置共享L3 Cache

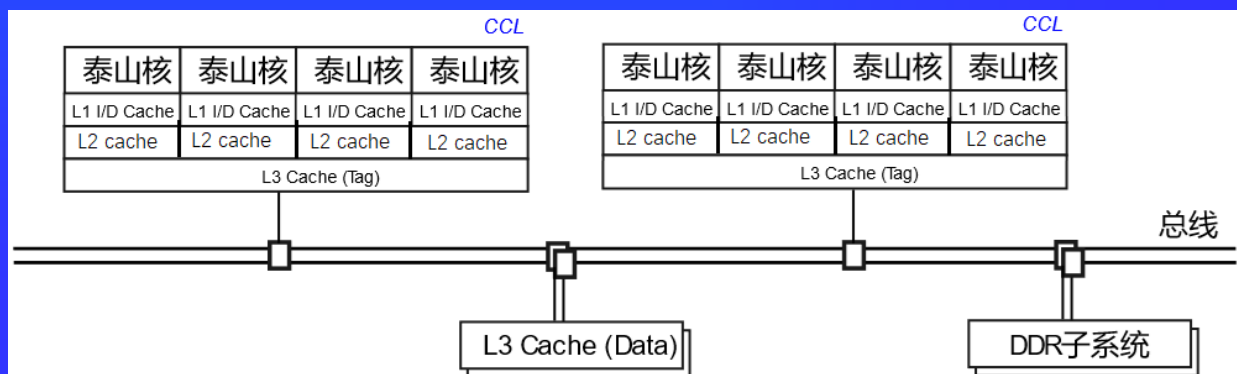
❑ 系统级片上三级高速缓存

❑ L3 Cache数据块：处在内核集群之外

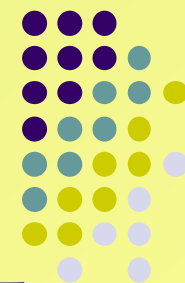
❑ L3 Cache标记块：协议处理模块，集成在每个内核集群中

» 降低监听延迟

❑ 主存

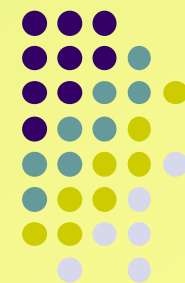


本章小结



- 对存储器的要求是容量大、速度快、成本低。
- 为了解决了这三方面的矛盾，计算机采用多级存储体系结构，即Cache、主存和外存。
- 内存储器的技术指标有存储容量、存取时间、存储周期、存储器带宽。
- SRAM、DRAM及ROM的相关特性。

本章小结



- 为提高内存使用效率有三种解决方法：
 - › 研究新的内存材料技术
 - › 使用并行技术：
 - ◆ 双端口存储器（空间并行）
 - ◆ 多模块交叉存储器（时间并行）
 - › 使用多层次存储体系
 - ◆ **Cache**
 - 命中率计算
 - 映射（全相联、直接映射、组相联）
 - 替换方法、写操作策略、多层**Cache**
 - ◆ 虚存

作业题:

● P122-123

9、10、13、14、15、21